

ĐIỆN TOÁN ĐÁM MÂY

Chương 3 – Lưu trữ, cơ sở dữ liệu và xử lý dữ liệu

Số tín chỉ: 3

Giảng viên: TS. Lê Quốc Tuấn
Email : Tuan.le@ut.edu.vn

Tình huống mở đầu: dữ liệu tăng nhanh

- Một hệ thống thương mại điện tử tạo log, đơn hàng, ảnh sản phẩm, hành vi người dùng mỗi ngày
- Dữ liệu cần lưu trữ an toàn, truy vấn nhanh, mở rộng linh hoạt và tối ưu chi phí
- **Câu hỏi:** chọn lưu trữ, cơ sở dữ liệu và pipeline xử lý như thế nào?



Viễn cảnh Cloud - AI

Mục tiêu học tập

- Phân biệt được dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc
- So sánh block, file và object storage
- Mô tả nguyên lý NFS, AFS, GFS, HDFS và lưu trữ phân tán
- Lựa chọn SQL, NoSQL, data warehouse, data lake hoặc lakehouse phù hợp
- Giải thích pipeline xử lý dữ liệu lớn trên cloud



**Mục tiêu
học tập**

Tổng quan về cơ sở hạ tầng đám mây với luồng dữ liệu



Sơ đồ kiến trúc dữ liệu đám mây



Vai trò của dữ liệu trong kiến trúc cloud

- Ứng dụng có thể khởi tạo lại nhanh, nhưng dữ liệu thường là tài sản khó thay thế
- Dữ liệu quyết định độ tin cậy, chi phí, bảo mật và khả năng phân tích
- Thiết kế dữ liệu sai có thể gây nghẽn hiệu năng hoặc chi phí tăng mạnh
- Quản trị dữ liệu cần gắn với bảo mật, sao lưu và vòng đời dữ liệu

1. NHỮNG VẤN ĐỀ CƠ BẢN VỀ DỮ LIỆU ĐÁM MÂY

- Dữ liệu, độ trễ, thông lượng, độ bền, nhất quán và CAP



Dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc

- Dữ liệu có cấu trúc: bảng, hàng, cột, khóa, quan hệ
- Dữ liệu bán cấu trúc: JSON, XML, log có mẫu nhưng linh hoạt
- Dữ liệu phi cấu trúc: ảnh, video, âm thanh, tài liệu, email
- Mỗi loại dữ liệu phù hợp với công nghệ lưu trữ và truy vấn khác nhau

Có cấu trúc
SQL

Bán cấu trúc
JSON/XML

Phi cấu trúc
ảnh, log, video

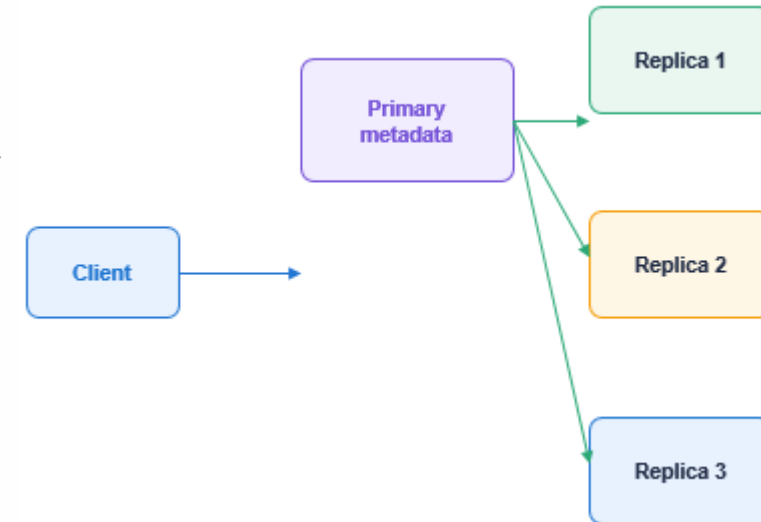
Dữ liệu đám mây cần quản lý: dung lượng • độ trễ • thông lượng • độ bền • quyền truy cập

Ba chỉ số kỹ thuật quan trọng

- Dung lượng: tổng dữ liệu cần lưu trữ và tăng trưởng theo thời gian
- Thông lượng: lượng dữ liệu đọc/ghi trong một đơn vị thời gian
- Độ trễ: thời gian phản hồi cho một yêu cầu đọc/ghi
- Một hệ thống có thể cần dung lượng lớn nhưng không nhất thiết cần độ trễ thấp

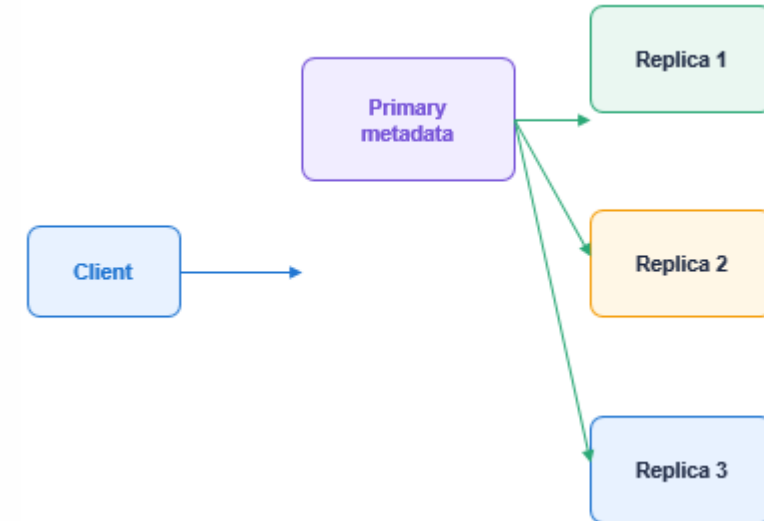
Độ bền và tính sẵn sàng dữ liệu

- Độ bền dữ liệu phản ánh khả năng không mất dữ liệu theo thời gian
- Tính sẵn sàng phản ánh khả năng truy cập dịch vụ khi cần
- Độ bền thường đạt bằng nhân bản, mã xóa và sao lưu
- Tính sẵn sàng thường đạt bằng triển khai nhiều vùng hoặc nhiều nút



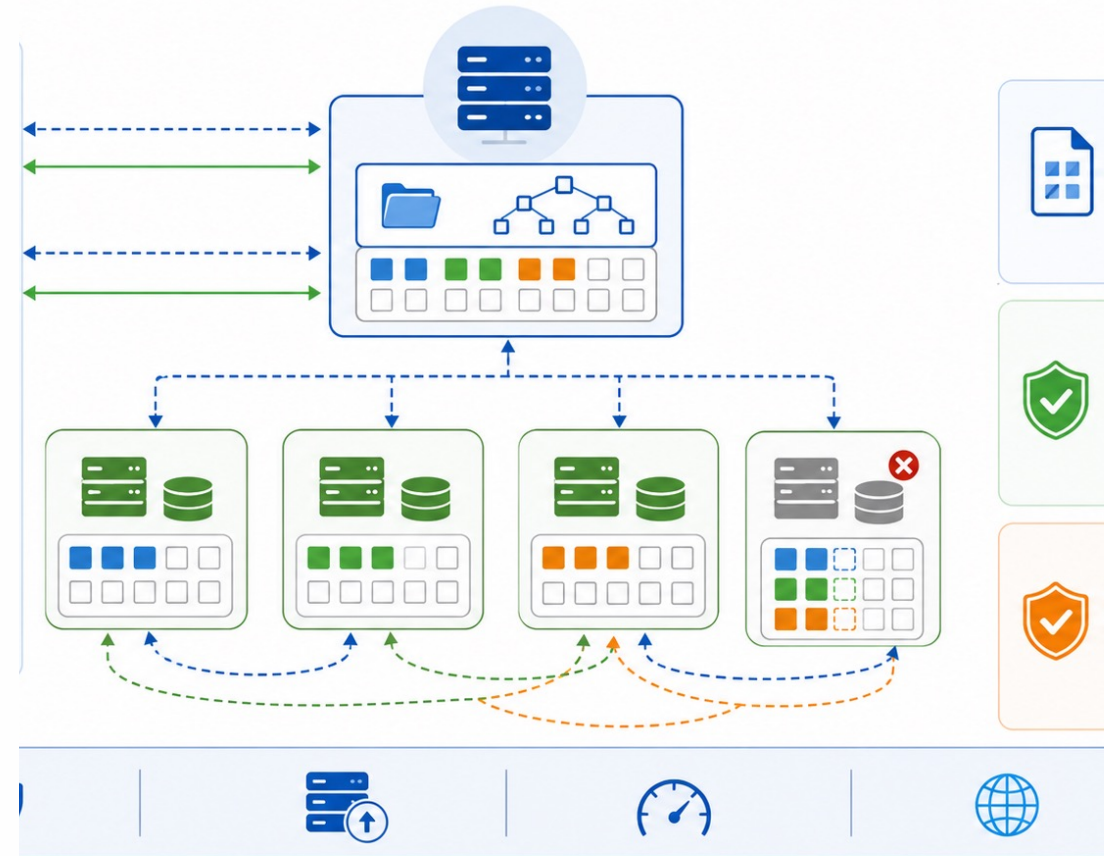
Nhân bản dữ liệu

- Sao chép dữ liệu sang nhiều nút, nhiều vùng sẵn sàng hoặc nhiều region
- Tăng khả năng chịu lỗi và đọc dữ liệu
- Có thể gây chi phí cao hơn và độ trễ đồng bộ
- Cần xác định chiến lược đồng bộ đồng bộ hoặc bất đồng bộ



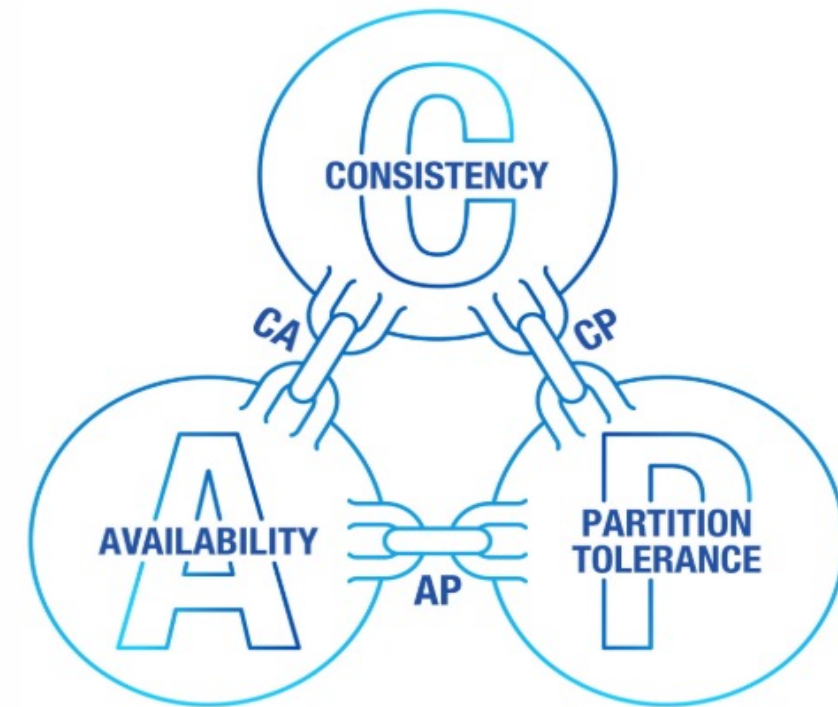
Phân vùng dữ liệu

- Chia dữ liệu thành nhiều phần để lưu trữ và xử lý song song
- Có thể phân vùng theo khóa, thời gian, vùng địa lý hoặc loại dữ liệu
- Giúp mở rộng ngang và giảm tải trên một nút
- Thiết kế khóa phân vùng kém có thể gây hot partition



Tính nhất quán dữ liệu

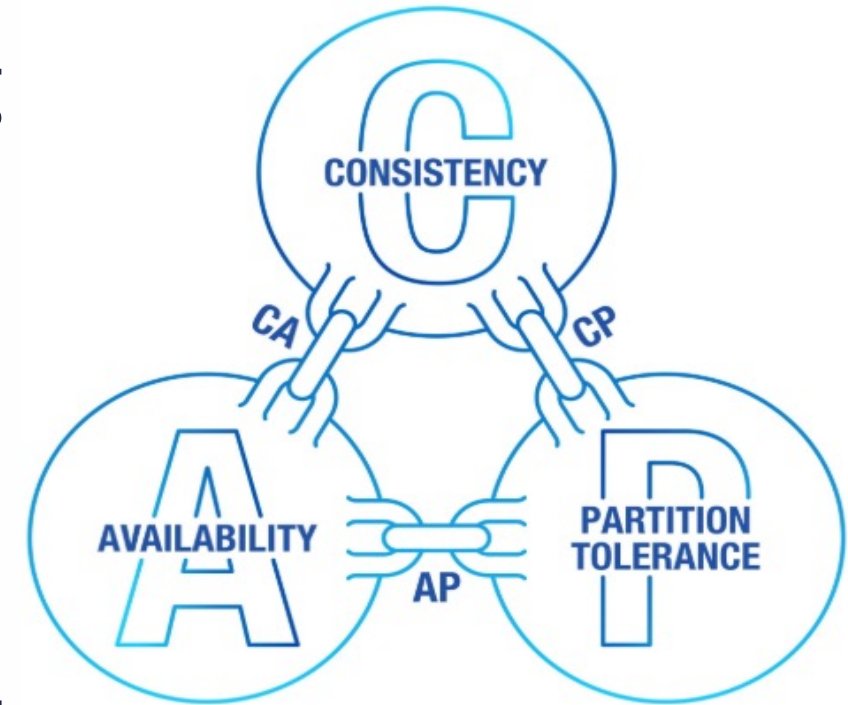
- Nhất quán mạnh: mọi lần đọc đều thấy bản ghi mới nhất
- Nhất quán cuối cùng: dữ liệu sẽ đồng bộ sau một khoảng thời gian
- Nhất quán theo phiên: người dùng thấy cập nhật của chính mình
- Mức nhất quán càng cao thường càng tốn chi phí và độ trễ



**Trong hệ phân tán
thiết kế dữ liệu luôn phải đánh đổi**

Định lý CAP trong hệ phân tán

- Consistency: các nút nhìn thấy cùng một dữ liệu
- Availability: mọi yêu cầu hợp lệ đều nhận phản hồi
- Partition tolerance: hệ thống vẫn hoạt động khi mạng bị chia cắt
- Khi có phân vùng mạng, hệ thống thường phải ưu tiên C hoặc A



Tam giác Định lý CAP và
các hướng đánh đổi kiến trúc

Phân loại các Hệ cơ sở dữ liệu theo CAP

Hệ thống CP (Consistency + Partition Tolerance)

- **Cơ chế:** Khi xảy ra phân vùng mạng, nếu một node không thể đồng bộ hóa dữ liệu với các node khác, nó sẽ **từ chối xử lý request** (trả về error) để bảo vệ tính chính xác tuyệt đối của dữ liệu.
- **Hệ quản trị tiêu biểu:** MongoDB, HBase, Redis (Single-master cluster), CockroachDB, Google Cloud Spanner.
- **Bài toán thực tế:** Hệ thống Ngân hàng, Ví điện tử, Quản lý Số dư tài khoản, Đặt vé máy bay (Tuyệt đối không được phép xảy ra hiện tượng *Double-spending* hay hạch toán sai).

Phân loại các Hệ cơ sở dữ liệu theo CAP

Hệ thống AP (Availability + Partition Tolerance)

- **Cơ chế:** Khi xảy ra phân vùng mạng, tất cả các node vẫn tiếp tục chấp nhận request, đọc/ghi. Dữ liệu giữa các node có thể bị lệch nhau tạm thời, nhưng hệ thống sẽ tự động đồng bộ lại khi đường truyền mạng phục hồi (**Eventual Consistency** - Nhất quán sau cùng).
- **Hệ quản trị tiêu biểu:** Apache Cassandra, Amazon DynamoDB, CouchDB, Riak.
- **Bài toán thực tế:** Mạng xã hội (Bảng tin Facebook/Twitter), Hệ thống đếm lượt view/like, Giỏ hàng thương mại điện tử, Log hệ thống (Chấp nhận hiển thị chậm hoặc sai lệch vài giây nhưng không bao giờ được nghìn/sập).

Phân loại các Hệ cơ sở dữ liệu theo CAP

Hệ thống CA (Consistency + Availability - Non-Distributed)

- **Cơ chế:** Đảm bảo dữ liệu nhất quán và sẵn sàng cao, nhưng **chỉ hoạt động trên một Node duy nhất** (Single Node) hoặc cụm không hỗ trợ phân vùng mạng.
- **Hệ quản trị tiêu biểu:** RDBMS truyền thống chạy Single-instance (MySQL, PostgreSQL, Oracle Server).

Sự mở rộng của CAP: Định lý PACELC

- Định lý CAP chỉ mô tả hành vi hệ thống **khi có sự cố phân vùng mạng (Partition)**. Để bù đắp thiếu sót này, nhà khoa học Daniel Abadi đã đề xuất **Định lý PACELC** bao hàm cả trạng thái bình thường:

If (Partition): Choose (A or C), Else (E): Choose (L or C)

- **P / A / C**: Nếu có **Partition**, chọn giữa **Availability** hay **Consistency** (Như CAP truyền thống).
- **E / L / C**: **Else** (Khi hệ thống hoạt động bình thường), chọn giữa **Latency** (Giảm độ trễ) hay **Consistency** (Nhất quán dữ liệu).
- *Ví dụ*: **Cassandra** là hệ thống **PA/EL** (Ưu tiên Sẵn sàng khi lỗi và Ưu tiên Tối ưu độ trễ khi bình thường); trong khi **MongoDB** là **PC/EC** (Luôn ưu tiên Nhất quán dữ liệu trong mọi trường hợp).

Bài học thiết kế từ CAP

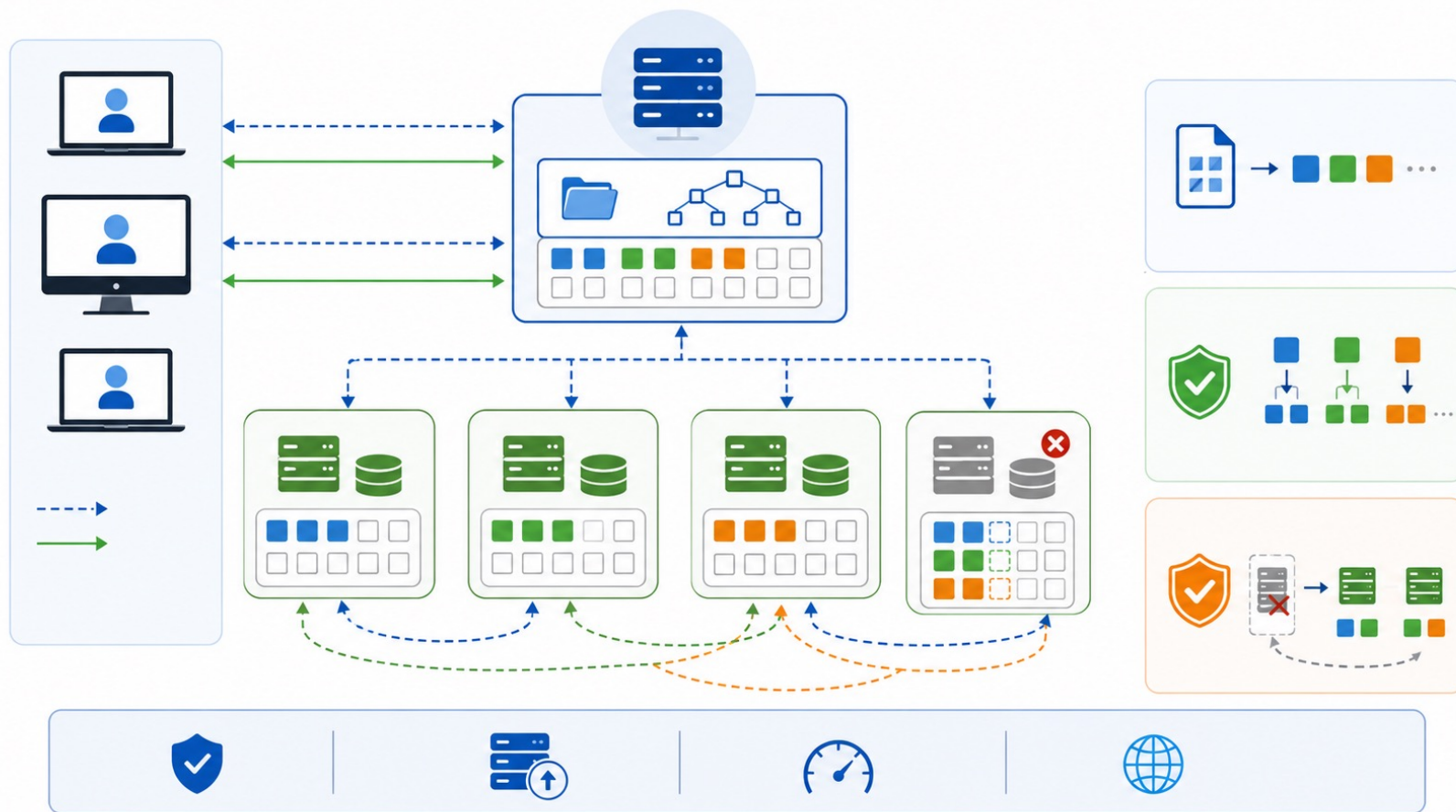
- Không có một cơ sở dữ liệu hoàn hảo cho mọi bài toán
- Giao dịch tài chính thường ưu tiên nhất quán
- Mạng xã hội, đo lường, log có thể chấp nhận nhất quán cuối cùng
- Thiết kế dữ liệu phải bắt đầu từ yêu cầu nghiệp vụ

Tiêu chí	SQL	NoSQL
Mô hình	Bảng, quan hệ	Linh hoạt
Schema	Chặt chẽ	Linh hoạt
Giao dịch	Mạnh	Tùy hệ
Mở rộng	Dọc/đọc replica	Ngang tốt
Dữ liệu	Có cấu trúc	Đa dạng

Bảng tóm tắt Quyết định Thiết kế theo CAP

Kịch bản Nghiệp vụ	Lựa chọn CAP	Chiến lược Kỹ thuật đề xuất
Giao dịch Tiền tệ / Chuyển khoản	CP (Consistency)	Database ACID/Spanner, Distributed Lock, Retry Mechanism, Circuit Breaker.
Bảng tin Social / Timeline / Feeds	AP (Availability)	NoSQL (Cassandra/DynamoDB), Eventual Consistency, Caching (Redis), Message Queue.
Hệ thống IoT / Streaming Telemetry	AP (Availability)	Time-series DB, Async Write, Ingest Buffer (Kafka), Chấp nhận drop gói tin lỗi.
Đặt chỗ / Đặt vé Máy bay	CP (Consistency)	Pessimistic Locking, Two-Phase Commit (2PC), Saga Pattern.

2. HỆ THỐNG TẬP MẠNG VÀ HỆ THỐNG TẬP PHÂN TÁN



Sơ đồ kiến trúc hệ thống tập tin phân tán

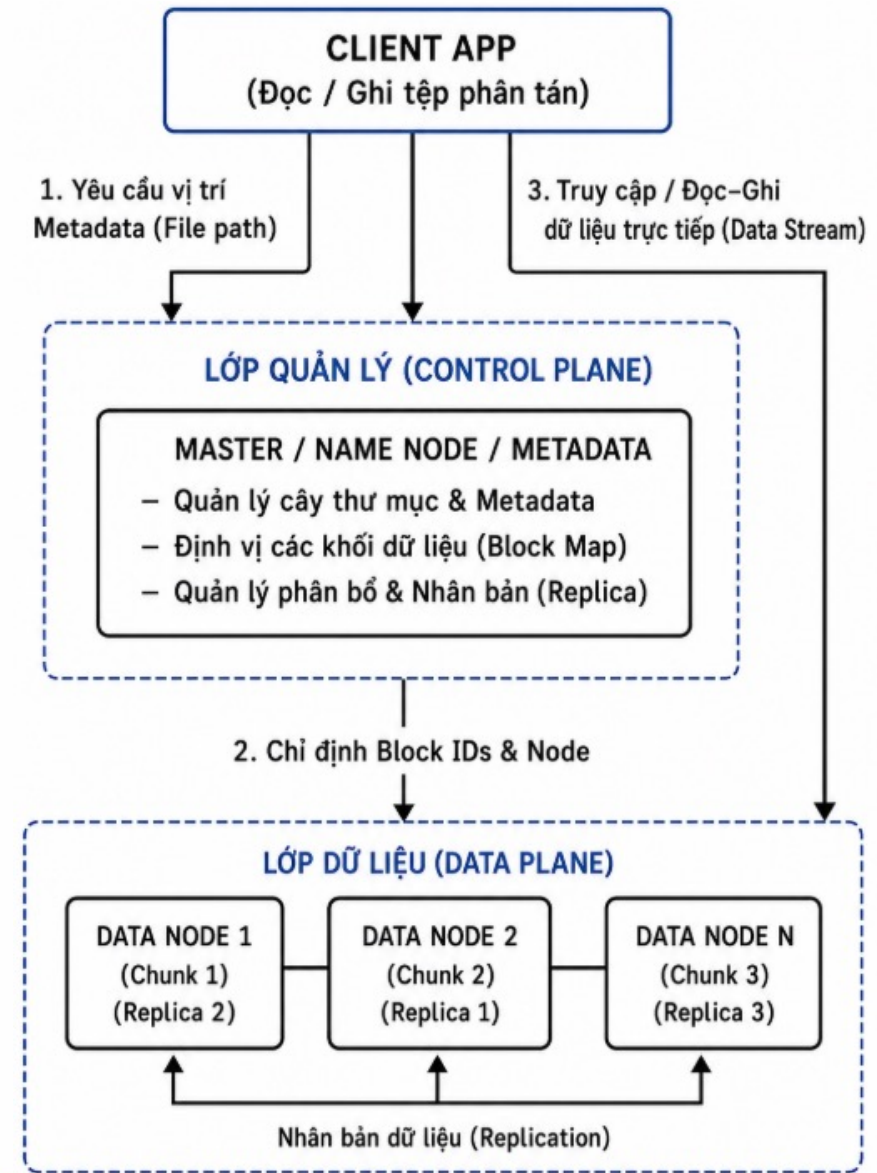
Từ hệ thống tập cục bộ đến hệ thống tập phân tán

- Hệ thống tập tin cục bộ nằm trên một máy chủ hoặc một thiết bị lưu trữ
- Hệ thống tập tin mạng cho phép nhiều máy truy cập qua mạng
- Hệ thống tập phân tán chia dữ liệu trên nhiều nút
- Mục tiêu: mở rộng dung lượng, tăng sẵn sàng và xử lý song song

Kiến File System - trúc hệ thống tệp phân tán (Distributed DFS) trong điện toán đám mây.

NFS hoạt động theo mô hình **Client - Server** truyền thống thông qua giao thức gọi thủ tục từ xa (**RPC - Remote Procedure Call**):

- NFS Server:** Mở (Share/Export) một hoặc nhiều thư mục trên ổ cứng của mình và thiết lập quyền truy cập (Đọc/Ghi, IP được phép kết nối).
- NFS Client:** Sử dụng lệnh Mount để gắn (Mount) thư mục mạng được chia sẻ từ NFS Server vào một đường dẫn/thư mục thuộc hệ thống tệp cục bộ.
- Người dùng / Ứng dụng:** Thao tác đọc, ghi, sửa, xóa file trên thư mục mount đó hoàn toàn minh bạch (Transparent) mà không cần quan tâm dữ liệu thực tế đang nằm trên máy chủ khác.



Các thành phần chính trong hệ thống tệp phân tán

Client (Ứng dụng phía người dùng)

- **Chức năng:** Giao tiếp với lớp quản lý để xin vị trí lưu trữ, sau đó trực tiếp đọc/ghi dữ liệu với các máy chủ chứa dữ liệu.
- **Cơ chế:** Không trực tiếp gửi dữ liệu qua nút Master để tránh gây nghẽn cổ chai (Bottleneck) hệ thống.

Các thành phần chính trong hệ thống tệp phân tán

Master / Name Node (Nút quản lý)

- **Chức năng:** Quản lý toàn bộ thông tin mô tả (**Metadata**) của hệ thống tệp.
- **Chi tiết công việc:**
 - Lưu trữ cấu trúc cây thư mục, quyền truy cập (ACL), thuộc tính tệp.
 - Bản đồ ánh xạ: Tệp đó được chia thành các **Khối (Block / Chunk)** nào và các khối đó nằm ở máy chủ dữ liệu nào.
 - Theo dõi tình trạng hoạt động (Health status/Heartbeat) của các Data Node.
 - Điều phối việc sao chép (Replication) dữ liệu khi có máy chủ bị hỏng.

Các thành phần chính trong hệ thống tệp phân tán

Data Node / Chunk Server (Các nút chứa dữ liệu)

- **Chức năng:** Lưu trữ các khối dữ liệu thực tế (thường có kích thước lớn, ví dụ: 64MB - 128MB mỗi block).
- **Chi tiết công việc:**
 - Thực hiện đọc/ghi các khối dữ liệu theo yêu cầu trực tiếp từ Client.
 - Định kỳ gửi báo cáo danh sách khối dữ liệu (Block Report) và tín hiệu nhịp tim (Heartbeat) về cho Master Node.
 - Tự động nhân bản các khối dữ liệu sang các Data Node khác theo chỉ thị từ Master Node.

Luồng hoạt động cơ bản (Read / Write Flow)

Luồng Đọc dữ liệu (Read Flow):

- **Client** gửi tên tệp cần đọc tới **Master Node**.
- **Master Node** tra cứu metadata và trả về danh sách các **Block ID** cùng địa chỉ các **Data Node** đang chứa các block đó.
- **Client** kết nối **trực tiếp** tới Data Node gần nhất (hoặc ít tải nhất) để tải dữ liệu về.

Luồng hoạt động cơ bản (Read / Write Flow)

Luồng Ghi dữ liệu (Write Flow):

- **Client** gửi yêu cầu tạo tệp mới tới **Master Node**.
- **Master Node** kiểm tra quyền, cấp phát không gian và trả về danh sách các **Data Node** sẽ lưu trữ khối dữ liệu (bao gồm Data Node chính và các Data Node phụ dùng để nhân bản).
- **Client** đẩy dữ liệu trực tiếp lên Data Node đầu tiên.
- **Data Node 1** ghi dữ liệu và tự động nối đường truyền (Pipeline) để nhân bản dữ liệu sang **Data Node 2** và **Data Node 3**.
- Sau khi hoàn tất việc nhân bản, các Data Node phản hồi lại trạng thái cho Client và Master Node.

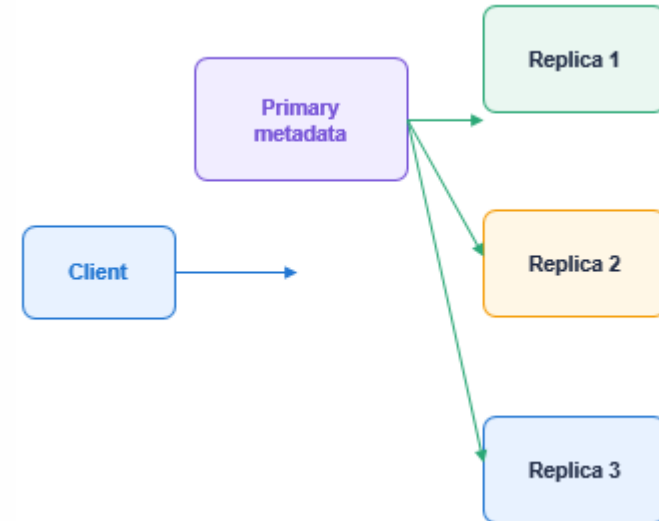
Các tính năng kỹ thuật cốt lõi

- **Phân chia khối dữ liệu (Data Chunking):** Tập dữ liệu lớn được xé nhỏ thành các khối cố định (như 64MB/128MB) để dễ dàng phân tán và xử lý song song.
- **Mức độ nhân bản (Replication Factor):** Mặc định mỗi khối dữ liệu sẽ được nhân bản thành 2 - 3 bản sao lưu ở các máy chủ/kệ máy chủ (Rack) khác nhau để chống mất dữ liệu khi hardware bị lỗi.
- **Khả năng chịu lỗi & Tự phục hồi (Fault Tolerance & Self-healing):** Khi một Data Node gặp sự cố (mất kết nối), Master Node phát hiện thông qua Heartbeat và ngay lập tức chỉ đạo các Data Node còn lại tạo bản sao bổ sung để đảm bảo đủ số lượng bản sao quy định.
- **Tránh nghẽn băng thông ở Master Node:** Tất cả lưu lượng dữ liệu nặng (Data Stream) đều truyền trực tiếp giữa Client và Data Node, Master Node chỉ xử lý các gói tin điều khiển (Control Stream) dung lượng nhỏ.

NFS – Network File System

Vai trò của NFS trong Điện toán đám mây (Cloud Computing)

- Trong môi trường Cloud, NFS đóng vai trò là nền tảng cho dịch vụ **Shared File Storage** (Lưu trữ tệp chia sẻ):
- **Chia sẻ dữ liệu giữa nhiều Máy chủ ảo (EC2 / Compute Engine):** Khác với Block Storage (như AWS EBS) thường chỉ gắn vào 1 VM tại 1 thời điểm, NFS cho phép **hàng trăm VM cùng kết nối, đọc và ghi vào một kho dữ liệu chung** đồng thời.
- **Các dịch vụ Cloud NFS tiêu biểu:**
 - **AWS:** Amazon EFS (Elastic File System).
 - **Google Cloud:** Cloud Filestore.
 - **Microsoft Azure:** Azure Files (hỗ trợ NFSv3).
- **Lưu trữ tĩnh cho Container / Kubernetes:** Được dùng làm **Persistent Volume (PV)** để lưu trữ dữ liệu bền vững cho các Pods trong Kubernetes cluster.



Ưu điểm và Hạn chế của NFS

- Ưu điểm
- Dễ chia sẻ: Cho phép nhiều máy chủ truy cập và chia sẻ dữ liệu chung một cách đồng bộ.
- Minh bạch (Transparent): Người dùng và phần mềm thao tác bằng các lệnh POSIX chuẩn (ls, cp, mv) mà không cần viết lại ứng dụng.
- Tiết kiệm chi phí lưu trữ: Tránh việc phải nhân bản dữ liệu ra từng máy chủ riêng lẻ.
- Cấu hình đơn giản: Được tích hợp sẵn trong nhân (Kernel) của hầu hết các hệ điều hành Linux.

Ưu điểm và Hạn chế của NFS

Hạn chế

- **Nghẽn băng thông mạng (Network Overhead):** Hiệu năng phụ thuộc hoàn toàn vào tốc độ và độ trễ (Latency) của hạ tầng mạng.
- **Rủi ro Single Point of Failure:** Nếu máy chủ NFS Server bị hỏng và không có cơ chế HA (High Availability), tất cả các máy Client sẽ bị ngắt kết nối hoặc treo ứng dụng.
- **Hiệu năng I/O thấp hơn Block Storage:** Không phù hợp cho các cơ sở dữ liệu (Database) yêu cầu tốc độ đọc/ghi ngẫu nhiên (Random I/O) siêu nhanh.

AFS – Andrew File System

- **AFS (Andrew File System)** là một hệ thống tệp phân tán (Distributed File System) tiên phong, được phát triển từ năm 1982 tại **Đại học Carnegie Mellon (CMU)** trong khuôn khổ dự án Andrew Project. AFS được thiết kế với mục tiêu cốt lõi: **Cho phép hàng chục nghìn máy trạm (Workstations) cùng chia sẻ một không gian lưu trữ tệp tin khổng lồ một cách an toàn, hiệu quả và có khả năng mở rộng cực cao.**
- Tên gọi "Andrew" được đặt theo tên của Andrew Carnegie và Andrew Mellon (hai nhà sáng lập CMU).

Các đặc trưng kỹ thuật đột phá của AFS

- Khác với NFS (Network File System) vốn phụ thuộc nhiều vào việc đọc/ghi liên tục qua mạng, AFS đưa ra nhiều cải tiến mang tính cách mạng thời bấy giờ:
-  Caching phía Client trên đĩa cứng (Whole-File Caching)
- Khi một ứng dụng trên Client mở một tệp, toàn bộ tệp tin đó sẽ được tải về và lưu vào bộ đệm (Cache) trên đĩa cứng cục bộ của Client.
- Mọi thao tác đọc/ghi tiếp theo đều diễn ra trực tiếp trên đĩa cục bộ, giúp tốc độ xử lý cực nhanh và giảm thiểu tối đa băng thông truyền tải qua mạng.
- Chỉ khi ứng dụng đóng tệp (close()), nếu tệp có sự thay đổi, bản cập nhật mới được gửi ngược lại Server.

Các đặc trưng kỹ thuật đột phá của AFS

Cơ chế Callbacks (Đảm bảo tính đồng bộ)

- Để giải quyết bài toán đồng bộ dữ liệu khi nhiều Client cùng cache một tệp: Khi Server giao một tệp cho Client, Server sẽ đăng ký một Callback cho Client đó.
- Nếu Client A sửa và đóng tệp, Server sẽ gửi một thông điệp "Break Callback" đến tất cả các Client khác đang giữ bản cache của tệp đó.
- Các Client nhận được thông điệp sẽ tự động vô hiệu hóa bản cache cũ và phải tải lại bản mới từ Server ở lần truy cập tiếp theo.

Các đặc trưng kỹ thuật đột phá của AFS

Không gian tên toàn cục thống nhất (Single Global Name Space)

- Bất kể người dùng đăng nhập vào máy trạm nào trong mạng, cấu trúc cây thư mục vẫn hoàn toàn giống nhau.
- Mọi tệp tin đều nằm dưới một cây thư mục gốc thống nhất (thường bắt đầu bằng đường dẫn /afs/...), giúp ẩn đi vị trí địa lý thực tế của Server chứa tệp.

Khái niệm Volume (Khối lưu trữ logic)

- AFS quản lý tệp theo các Volume (tập hợp các tệp/thư mục liên quan, nhỏ hơn một phân vùng đĩa).
- Các Volume có thể được di chuyển linh hoạt giữa các Server thực tế trong lúc hệ thống vẫn đang vận hành mà người dùng không hề bị gián đoạn hay nhận ra sự thay đổi.

Các đặc trưng kỹ thuật đột phá của AFS

Bảo mật mạnh mẽ với Kerberos & ACLs

- AFS tích hợp hệ thống xác thực tập trung Kerberos.
- Quản lý quyền truy cập chi tiết dựa trên danh sách kiểm soát truy cập (ACLs - Access Control Lists) gắn với từng thư mục, thay vì chỉ dựa vào phân quyền rwxrwxrwx truyền thống của Unix.

So sánh AFS và NFS (Network File System)

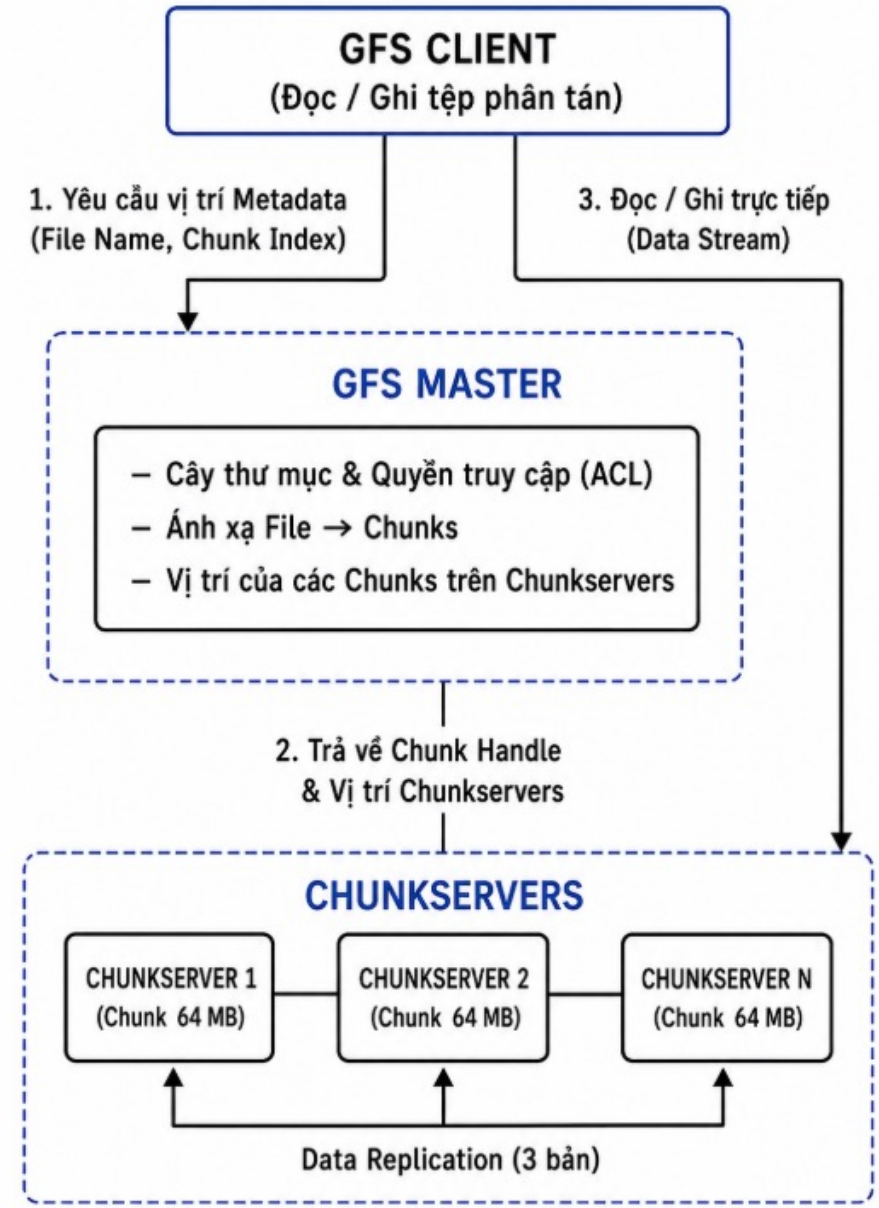
Tiêu chí	AFS (Andrew File System)	NFS (Network File System)
Cơ chế Caching	Cache toàn bộ tệp/khối lớn trên đĩa cứng Client	Cache các khối (Block) nhỏ trong RAM/Bộ nhớ tạm
Tải trên Server	Rất thấp (Server chỉ làm việc khi Open/Close file)	Cao (Server xử lý I/O cho mọi thao tác Read/Write)
Khả năng mở rộng	Siêu lớn (hỗ trợ hàng chục nghìn Clients)	Trung bình (hiệu năng giảm khi số Client quá lớn)
Cấu trúc đường dẫn	Toàn cục thống nhất (/afs/cell_name/...)	Phụ thuộc vào việc Mount thủ công trên từng Client
Bảo mật	Tích hợp Kerberos & ACL theo thư mục	Dựa trên UID/GID của Unix, Kerberos (từ NFSv4)
Trạng thái (State)	Stateful (Server quản lý Callbacks)	Stateless (NFSv2/v3) / Stateful (NFSv4)

GFS – Google File System

- **GFS (Google File System)** là hệ thống tệp phân tán độc quyền do Google nghiên cứu và phát triển (công bố bài báo khoa học nổi tiếng năm 2003). GFS được thiết kế nhằm đáp ứng nhu cầu xử lý khối lượng dữ liệu khổng lồ (Big Data) cho các ứng dụng tìm kiếm và chỉ mục của Google, trên một hạ tầng gồm hàng nghìn **máy chủ thương mại giá rẻ (commodity hardware)** thường xuyên gặp sự cố phần cứng.
- GFS chính là nền tảng trực tiếp truyền cảm hứng cho sự ra đời của **Hadoop Distributed File System (HDFS)** trong hệ sinh thái mã nguồn mở.

Kiến trúc thành phần của GFS

- Mô hình GFS tuân theo kiến trúc **Single Master - Multiple Chunkservers**:



Kiến trúc thành phần của GFS

GFS Master (Nút quản lý trung tâm)

- Chức năng: Quản lý toàn bộ thông tin Metadata của hệ thống (lưu trên RAM để có tốc độ truy xuất cực nhanh).
- **Quản lý:** Cấu trúc cây thư mục và phân quyền. Bản đồ ánh xạ từ Tập tin → Các Chunk Handle (Mã định danh duy nhất 64-bit).
- Vị trí thực tế của từng Chunk trên các Chunkserver (được cập nhật liên tục qua tín hiệu Heartbeat).
- **Bảo toàn dữ liệu Master:** Mọi thay đổi Metadata đều được ghi vào Operation Log (lưu trên đĩa cứng local và nhân bản sang máy sao lưu) và định kỳ tạo Checkpoint.

Kiến trúc thành phần của GFS

GFS Chunkservers (Máy chủ chứa khối dữ liệu)

- **Chức năng:** Lưu trữ các khối dữ liệu thực tế (**Chunks**) dưới dạng các file chuẩn trên hệ thống tệp cục bộ (như ext3).
- **Kích thước Chunk mặc định: 64 MB** (Rất lớn so với block size 4KB của hệ thống tệp thông thường). Việc dùng chunk lớn giúp giảm kích thước Metadata lưu trên Master và giảm kết nối TCP.
- **Nhân bản (Replication):** Mặc định mỗi Chunk được lưu thành 3 bản sao (**Replicas**) phân bố ở các rack khác nhau để đảm bảo an toàn dữ liệu.

Kiến trúc thành phần của GFS

GFS Client

- **Chức năng:** Thư viện (Library) cung cấp các API như open, close, read, write, append tích hợp trực tiếp vào ứng dụng.
- **Cơ chế:** Client tương tác với Master để xin vị trí lưu Chunk (Metadata), sau đó truyền nhận dữ liệu trực tiếp với Chunkservers mà không đi qua Master.3. Luồng xử lý dữ liệu: Đọc & Ghi (Append)

Luồng xử lý dữ liệu: Đọc & Ghi (Append)

Luồng Đọc (Read):

- Client tính toán Chunk Index dựa trên vị trí tệp (Offset) và gửi (File Name, Chunk Index) tới Master.
- Master trả về Chunk Handle và danh sách vị trí các Chunkservers đang giữ bản sao.
- Client lưu thông tin này vào bộ nhớ tạm (Cache Metadata) để dùng lại.
- Client kết nối trực tiếp đến Chunkserver gần nhất để tải dữ liệu.

Luồng xử lý dữ liệu: Đọc & Ghi (Append)

Luồng Ghi nối tiếp (Record Append - Tính năng đặc trưng):

- Client hỏi Master xem Chunkserver nào đang giữ vai trò **Primary** cho Chunk hiện tại.
- Master chỉ định một Primary và cấp quyền gia hạn (Lease). Master trả danh sách Primary và các Secondary Chunkservers cho Client.
- Client đẩy dữ liệu lên **tất cả các Chunkserver** theo cơ chế chuỗi (Data Pipeline) để tối ưu băng thông mạng.
- Sau khi tất cả đã nhận đủ dữ liệu, Client gửi lệnh Ghi tới **Primary**.
- Primary gán thứ tự thực thi (Sequence Number) cho thao tác ghi, thực hiện ghi cục bộ, rồi yêu cầu tất cả các **Secondary** ghi theo đúng thứ tự đó.
- **Primary** phản hồi thành công về cho Client sau khi nhận đủ xác nhận từ các **Secondary**.

GFS - Tầm ảnh hưởng lịch sử & Tương lai (Colossus)

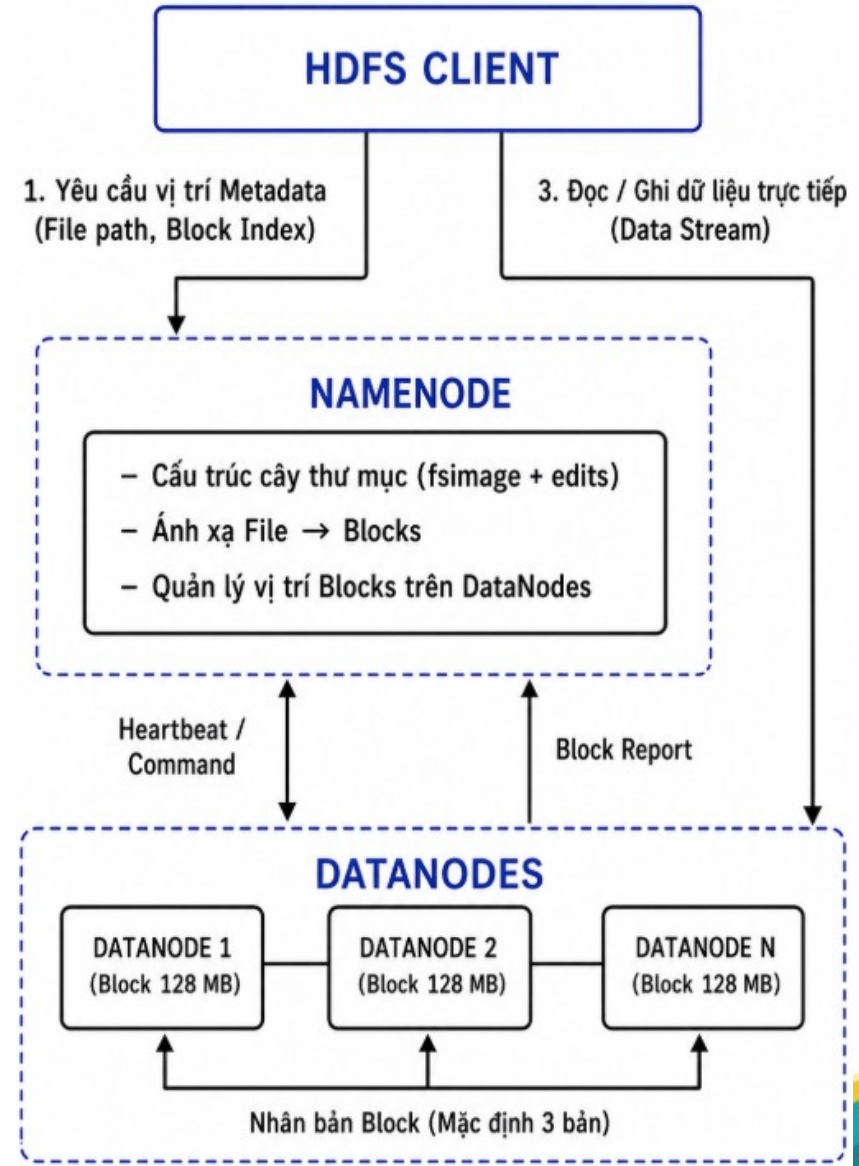
- **Nền tảng Big Data:** Thiết kế của GFS kết hợp cùng mô hình tính toán MapReduce và cơ sở dữ liệu Bigtable đã hình thành nên bộ ba công nghệ trụ cột làm thay đổi ngành xử lý dữ liệu lớn thế giới.
- **Sự kế thừa (Colossus / GFS2):** Do hạn chế của việc chỉ có Single Master (gây giới hạn dung lượng RAM chứa Metadata khi số lượng tệp lên tới hàng chục triệu) và kích thước Chunk 64MB chưa tối ưu cho tệp nhỏ, Google đã thay thế GFS bằng hệ thống tệp thế hệ tiếp theo mang tên Colossus (GFS2) với khả năng phân tán cả nút Master và hỗ trợ lưu trữ tối ưu hơn.

HDFS – Hadoop Distributed File System

- **Hadoop Distributed File System (HDFS)** là hệ thống tệp phân tán mã nguồn mở thuộc hệ sinh thái Apache Hadoop, được thiết kế để lưu trữ các tập dữ liệu cực lớn (Petabytes/Exabytes) và xử lý chúng trên các cụm máy chủ thương mại giá rẻ (*commodity hardware*).
- HDFS được xây dựng dựa trên nguyên mẫu bài báo khoa học về **Google File System (GFS)** do Google công bố năm 2003, với triết lý cốt lõi: "*Di chuyển tính toán đến dữ liệu rẻ hơn nhiều so với di chuyển dữ liệu đến tính toán*" (Data Locality).

Kiến trúc các thành phần trong HDFS

- HDFS vận hành theo kiến trúc **Master-Worker** (**NameNode** - **DataNodes**):



Ưu điểm và Hạn chế

Ưu điểm:

- **Khả năng mở rộng tuyến tính (Scalability):** Dễ dàng mở rộng từ vài máy chủ lên hàng nghìn máy chủ chỉ bằng cách cắm thêm DataNode mới vào cụm.
- **Khả năng chịu lỗi cao (Fault Tolerance):** Tự động phát hiện và khôi phục dữ liệu bị hỏng/mất nhờ chính sách nhân bản Block.
- **Chi phí rẻ:** Chạy tốt trên các phần cứng phổ thông thương mại.

Hạn chế:

- **Vấn đề tệp nhỏ (Small File Problem):** HDFS rất kém hiệu quả khi lưu hàng triệu tệp kích thước nhỏ (vài KB), vì mỗi tệp đều tiêu tốn một lượng RAM cố định trên NameNode để lưu Metadata, gây tràn bộ nhớ NameNode.
- **Độ trễ cao (High Latency):** Không phù hợp cho các ứng dụng yêu cầu truy xuất dữ liệu thời gian thực (Real-time Transaction / OLTP).
- **Chỉ hỗ trợ Append:** Không hỗ trợ ghi đè ngẫu nhiên vào giữa tệp (Random Write).

So sánh HDFS và GFS

Tiêu chí	HDFS (Hadoop)	GFS (Google)
Bản chất	Mã nguồn mở (Open-source)	Độc quyền của Google (Proprietary)
Ngôn ngữ phát triển	Java	C++
Thuật ngữ Nút quản lý	NameNode	Master
Thuật ngữ Nút dữ liệu	DataNode	Chunkserver
Kích thước Block/Chunk	128 MB (mặc định HDFS v2+)	64 MB (mặc định)
Cơ chế dự phòng Nút quản lý	HDFS HA (Active/Standby NameNode)	Shadow Masters (Chỉ đọc)

Khả năng chịu lỗi trong hệ tệp phân tán

- **Khả năng chịu lỗi (Fault Tolerance)** trong hệ thống tệp phân tán (Distributed File System - DFS) là khả năng duy trì hoạt động liên tục, đảm bảo **tính sẵn sàng (Availability)** và **tính toàn vẹn dữ liệu (Data Integrity)** ngay cả khi một hoặc nhiều thành phần phần cứng (ổ đĩa, nút máy chủ, cổng mạng, kệ máy chủ) hoặc phần mềm bị hỏng hóc.
- Để đạt được khả năng chịu lỗi tối ưu, một hệ thống tệp phân tán thường kết hợp nhiều cơ chế kỹ thuật từ mức lưu trữ, giao tiếp đến quản lý trạng thái.

Khả năng chịu lỗi trong hệ tệp phân tán

- Cơ chế dự phòng & Nhân bản dữ liệu (Data Redundancy)
- Mã hóa xóa (Erasure Coding - EC)
- Cơ chế phát hiện lỗi & Tự phục hồi (Detection & Self-healing)
- Kiểm tra tính toàn vẹn dữ liệu (Data Integrity Check)
- Bảo vệ Nút quản lý trung tâm (Master / NameNode Fault Tolerance)
- Cơ chế xử lý tính đồng bộ & Độc quyền (Consistency & Locking)

Khả năng chịu lỗi trong hệ tệp phân tán

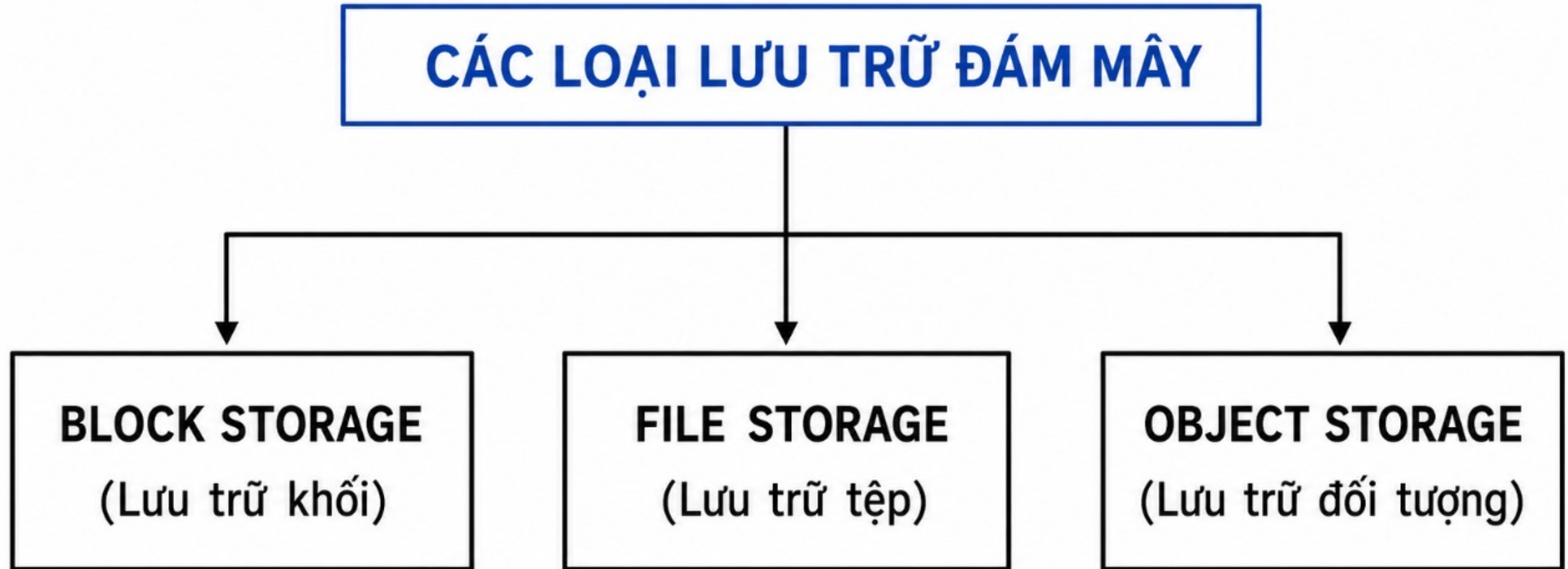
- **Tóm lại:** Khả năng chịu lỗi trong hệ tệp phân tán không dựa vào việc "ngăn chặn phần cứng bị hỏng", mà dựa trên nguyên tắc "**thừa nhận phần cứng chắc chắn sẽ hỏng**" để thiết kế các lớp bảo vệ tự động: từ Dự phòng dữ liệu (Replication/EC), Phát hiện sự cố (Heartbeat) đến Tự phục hồi (Self-healing) và Bảo vệ Metadata (HA Architecture).

Tiêu chí	NFS (Network File System)	AFS (Andrew File System)	GFS (Google File System)	HDFS (Hadoop DFS)
Mục đích thiết kế chính	Chia sẻ tệp cục bộ qua mạng giống như đĩa cứng thật	Chia sẻ tệp quy mô lớn cho hàng nghìn máy trạm (Workstations)	Lưu trữ & xử lý Big Data trên máy chủ thương mại giá rẻ	Lưu trữ Big Data mã nguồn mở cho hệ sinh thái Hadoop
Kiến trúc (Architecture)	Client - Server	Client - Server (Volume-based)	Single Master - Multiple Chunkservers	NameNode (Master) - DataNodes (Workers)
Kích thước khối (Block/Chunk)	Nhỏ (4 KB - 64 KB)	Nhỏ / Lưu toàn bộ tệp (Whole-file)	Rất lớn (64 MB)	Rất lớn (128 MB / 256 MB)
Cơ chế Caching	Cache khối dữ liệu trên RAM/Bộ nhớ tạm Client	Whole-file Caching trên đĩa cứng Client	Metadata Cache ở Client; Dữ liệu truyền trực tiếp qua Stream	Metadata Cache ở Client; Dữ liệu truyền dạng Pipeline Stream
Khả năng mở rộng (Scalability)	Thấp - Trung bình	Cao (Hàng chục nghìn Client)	Rất cao (Hàng triệu tệp/Petabytes)	Siêu lớn (Hàng Petabytes/Exabytes)
Mẫu dữ liệu ưu tiên (Access Pattern)	Đọc/Ghi ngẫu nhiên (Random Read/Write)	Đọc/Ghi tập trung tại Client khi Open/Close file	Đọc luồng lớn & Ghi nối tiếp (Append-only)	Đọc luồng lớn & Ghi 1 lần (Write Once, Read Many)
Cơ chế chịu lỗi (Fault Tolerance)	Phụ thuộc vào hạ tầng lưu trữ Server (RAID/HA)	Dự phòng nút lưu trữ / Quản lý Volume	Nhân bản Chunk (Replication - 3 bản)	Nhân bản Block (Replication - 3 bản) / Erasure Coding
Bảo mật & Phân quyền	POSIX permissions, UID/GID, Kerberos (NFSv4)	Tích hợp Kerberos & ACL theo thư mục	Định danh nội bộ của Google	POSIX-like permissions, Kerberos, ACLs
Tuân thủ chuẩn POSIX	Có (đầy đủ)	Gần như đầy đủ	Không (Dùng API riêng: read, write, append)	Không (Dùng Java API / HDFS CLI riêng)

So sánh NFS, AFS, GFS và HDFS

- **Chọn NFS khi:** Cần một hệ thống lưu trữ tệp dùng chung đơn giản, tuân thủ POSIX chuẩn, phù hợp cho hệ thống ứng dụng Web, chứa code, chia sẻ file nội bộ nhỏ và vừa.
- **Chọn AFS khi:** Cần một không gian tên toàn cục (Global Namespace) thống nhất cho hàng nghìn máy trạm phân tán địa lý với yêu cầu bảo mật Kerberos cao.
- **Chọn GFS / HDFS khi:** Xây dựng kho dữ liệu Big Data (Data Lake), lưu trữ log tập trung, phân tích dữ liệu lớn, nơi dữ liệu chủ yếu là tệp dung lượng không lồ được ghi nối tiếp (Append) và xử lý bằng các công cụ như Hadoop, Spark, Hive.

3. CÁC LOẠI LƯU TRỮ ĐÁM MÂY



PHÂN LOẠI THEO CẤU TRÚC DỮ LIỆU & GIAO THỨC TRUY CẬP
(3 Mô hình cốt lõi)

3. CÁC LOẠI LƯU TRỮ ĐÁM MÂY

- Block storage: đĩa ảo gắn cho máy ảo hoặc CSDL
- File storage: thư mục chia sẻ qua giao thức tệp
- Object storage: lưu đối tượng trong bucket, truy cập qua API
- Cần chọn theo cách ứng dụng đọc/ghi dữ liệu

1. Block Storage (Lưu trữ dạng Khối)

- **Bản chất:** Chia nhỏ dữ liệu thành các khối cứng (blocks) có kích thước cố định, mỗi khối được cấp một địa chỉ duy nhất. Nó hoạt động tương tự như một ổ cứng trống (HDD/SSD) được gắn trực tiếp vào máy chủ ảo.
- **Giao thức truy cập:** iSCSI, Fibre Channel, NVMe-oF.
- **Đặc tính:** Độ trễ cực thấp (Low Latency), hiệu năng I/O (IOPS) siêu cao.
 - ✓ Hỗ trợ định dạng hệ thống tệp tùy chỉnh (ext4, NTFS) và cho phép ghi/sửa dữ liệu ngẫu nhiên từng byte.
 - ✓ Thường chỉ gắn được vào 1 máy chủ ảo tại một thời điểm (trừ một số cấu hình Multi-attach đặc thù).
- **Trường hợp sử dụng (Use-case):** Cơ sở dữ liệu (SQL, Oracle), lưu trữ hệ điều hành (Boot volumes), ứng dụng giao dịch thời gian thực (OLTP).
- **Dịch vụ tiêu biểu:** AWS EBS (Elastic Block Store), Google Persistent Disk, Azure Managed Disks.

2. File Storage (Lưu trữ dạng Tập)

- **Bản chất:** Tổ chức dữ liệu theo cây thư mục phân cấp chuẩn (Folders/Sub-folders/Files). Cho phép nhiều máy chủ ảo cùng kết nối, đọc và ghi dữ liệu đồng thời.
- **Giao thức truy cập:** NFS (cho Linux/Unix), SMB/CIFS (cho Windows).
- **Đặc tính:** Thân thiện với ứng dụng truyền thống, tuân thủ tiêu chuẩn POSIX.
 - ✓ Người dùng và phần mềm thao tác như với một ổ đĩa mạng dùng chung (Network Drive).
 - ✓ Hiệu năng I/O trung bình, phụ thuộc vào băng thông mạng nội bộ.
- **Trường hợp sử dụng (Use-case):** Hệ thống CMS (WordPress, Drupal) cần chung thư mục source code, lưu trữ tài liệu dùng chung doanh nghiệp, lưu trữ môi trường phát triển (Dev Environment), Kubernetes Persistent Volume.
- **Dịch vụ tiêu biểu:** AWS EFS (Elastic File System), Google Cloud Filestore, Azure Files.

3. Object Storage (Lưu trữ dạng Đối tượng)

- **Bản chất:** Lưu trữ dữ liệu dưới dạng các "đối tượng" (Objects) độc lập nằm trong một không gian phẳng (Flat Namespace - không phân cấp thư mục). Mỗi Object gồm 3 phần: **Dữ liệu thực tế + Metadata phong phú + ID định danh duy nhất.**
- **Giao thức truy cập:** RESTful API (HTTP/HTTPS) qua các lệnh GET, PUT, POST, DELETE.
- **Đặc tính:** Khả năng mở rộng gần như vô hạn (Petabytes đến Exabytes) với chi phí cực rẻ.
 - ✓ Dữ liệu là dạng không thể sửa đổi trực tiếp (Immutable) — muốn sửa phải ghi đè một phiên bản Object mới.
 - ✓ Không thể cắm làm ổ đĩa chạy OS hay Database.
- **Trường hợp sử dụng (Use-case):** Kho dữ liệu khổng lồ (Data Lake), sao lưu & khôi phục (Backup & Disaster Recovery), lưu trữ phương tiện truyền thông (Hình ảnh, Video, Audio), phân phối nội dung tĩnh qua CDN.
- **Dịch vụ tiêu biểu:** AWS S3, Google Cloud Storage, Azure Blob Storage.

Bảng so sánh nhanh 3 loại lưu trữ cốt lõi

Tiêu chí	Block Storage	File Storage	Object Storage
Cấu trúc	Khối dữ liệu thô	Cây thư mục phân cấp	Không gian phẳng (Flat)
Giao thức	iSCSI, Fibre Channel	NFS, SMB/CIFS	REST API (HTTP/HTTPS)
Hiệu năng (I/O)	Super High (Rất cao)	Medium (Trung bình)	High Throughput / High Latency
Khả năng chia sẻ	1-to-1 (Thường 1 VM)	Many-to-Many (Nhiều VM)	Internet-wide (Bất kỳ đâu qua API)
Chi phí / GB	Mặc nhất	Trung bình	Rẻ nhất
Độ linh hoạt Metadata	Hạn chế	Hạn chế (Chuẩn POSIX)	Rất phong phú (Custom Metadata)

PHÂN LOẠI THEO MÔ HÌNH TRIỂN KHAI (Deployment Models)

Nếu xét theo hình thức hạ tầng sở hữu và quyền quản lý, lưu trữ đám mây gồm 4 loại:

- **Public Cloud Storage (Đám mây công cộng):** Dữ liệu lưu trên hạ tầng của các nhà cung cấp lớn (AWS, GCP, Azure, Viettel, FPT). Chi phí linh hoạt theo mức sử dụng, không tốn phí bảo trì hạ tầng.
- **Private Cloud Storage (Đám mây riêng):** Hạ tầng lưu trữ được dựng riêng cho một doanh nghiệp (On-premise) sử dụng các công nghệ đám mây như OpenStack, Ceph, MinIO. Đảm bảo an toàn tuyệt đối và tuân thủ pháp lý cao.
- **Hybrid Cloud Storage (Đám mây hỗn hợp):** Kết hợp giữa Private và Public Cloud. Dữ liệu nhạy cảm lưu ở Private Cloud, dữ liệu tĩnh hoặc cần mở rộng linh hoạt đẩy lên Public Cloud.
- **Multi-Cloud Storage (Đa đám mây):** Sử dụng dịch vụ lưu trữ của từ 2 nhà cung cấp Public Cloud trở lên (ví dụ: dùng AWS S3 lưu dữ liệu chính, dùng Google Cloud Storage làm Backup) nhằm tránh rủi ro khóa chặt nhà cung cấp (*Vendor Lock-in*).

PHÂN LOẠI THEO MÀU TẦN SUẤT TRUY CẬP (Storage Tiers / Classes)

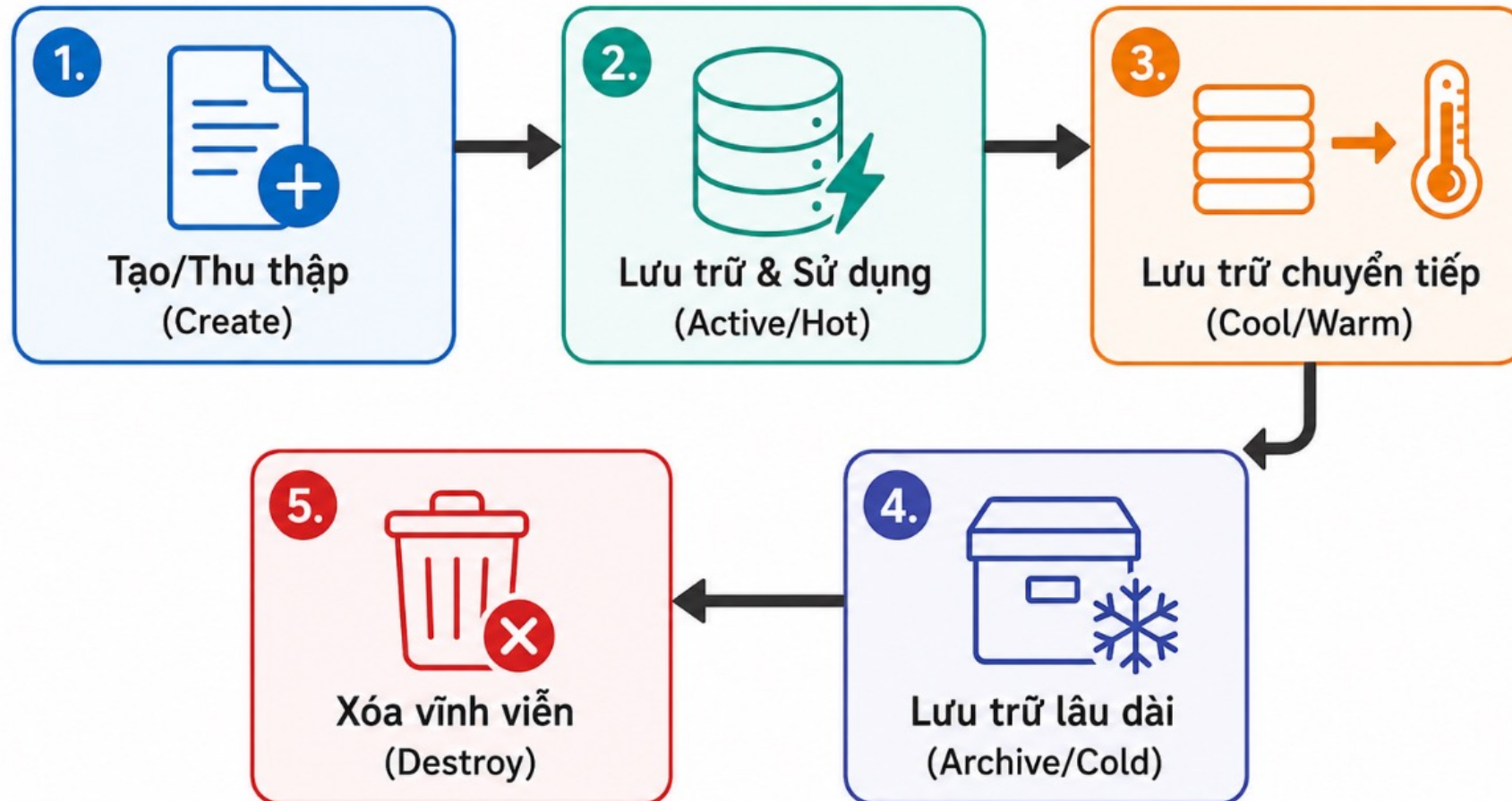
Đối với **Object Storage**, các nhà cung cấp đám mây còn chia nhỏ thành các **tầng lưu trữ (Tiers)** dựa trên tần suất truy xuất để tối ưu chi phí cho người dùng:

- **Hot Storage (Standard):** Dành cho dữ liệu truy cập thường xuyên hàng ngày (truy xuất tức thì, phí lưu trữ cao hơn, phí truy xuất rẻ).
- **Cool / Warm Storage:** Dành cho dữ liệu ít truy cập (1 tháng/lần) như báo cáo tháng, log hệ thống.
- **Cold / Archive Storage (Glacier):** Dành cho dữ liệu sao lưu dài hạn, hiếm khi truy cập (vài tháng hoặc vài năm/lần). Chi phí lưu trữ siêu rẻ, nhưng khi tải về sẽ mất vài phút đến vài giờ và tốn phí truy xuất.

Bucket, object và metadata

- Bucket là không gian chứa logic, là chiếc thùng chứa toàn cục
- Object là đơn vị dữ liệu được lưu trữ, là món hàng nằm trong thùng (chứa dữ liệu thực tế)
- Metadata giúp mô tả nội dung, loại, thời gian, nhãn và quyền, là tấm nhãn dán trên món hàng đó cung cấp toàn bộ thông tin chi tiết về nó.
- Object storage không phù hợp cho thao tác cập nhật nhỏ liên tục như CSDL giao dịch

Chính sách vòng đời dữ liệu



Các giai đoạn chính trong Vòng đời dữ liệu

1. **Tạo / Thu thập (Creation/Ingestion):** Dữ liệu được phát sinh từ ứng dụng, người dùng, thiết bị IoT hoặc nhật ký hệ thống (logs).
2. **Lưu trữ & Sử dụng (Active/Hot Storage):** Dữ liệu mới được truy cập thường xuyên với độ trễ thấp nhất.
3. **Chuyển tiếp / Giảm tần suất (Cool/Warm Storage):** Sau một thời gian (ví dụ: 30 ngày), dữ liệu ít được truy cập hơn nhưng vẫn cần sẵn sàng khi có yêu cầu.
4. **Lưu trữ lâu dài / Lưu trữ vết (Archive/Cold Storage):** Dữ liệu không còn được truy cập hàng ngày nhưng phải giữ lại để đối soát, tuân thủ pháp luật hoặc kiểm toán (ví dụ: giữ trong 5–10 năm).
5. **Tiêu hủy / Xóa vĩnh viễn (Destruction/Deletion):** Dữ liệu hết thời hạn lưu trữ quy định sẽ bị xóa an toàn và không thể phục hồi để giải phóng dung lượng và tránh rủi ro pháp lý.

Phân tầng lưu trữ

- Tầng truy cập thường xuyên: chi phí lưu cao hơn, truy xuất nhanh
- Tầng truy cập ít: lưu rẻ hơn, truy xuất có phí cao hơn
- Tầng lưu trữ lạnh: dùng cho backup và lưu trữ dài hạn
- Lựa chọn tầng phụ thuộc tần suất truy cập và RTO

Phân tầng lưu trữ



Hot

Truy cập liên tục / Tốc độ cực nhanh / Chi phí cao



Warm / Cool

Truy cập hàng tháng / Độ trễ thấp / Chi phí trung bình



Cold

Truy cập hàng năm / Độ trễ vài phút-giờ / Chi phí rẻ



Archive

Lưu trữ lâu dài / Độ trễ nhiều giờ / Chi phí siêu rẻ

Phân tầng lưu trữ

Tiêu chí	Hot (Standard)	Warm (Cool / Nearline)	Cold (Coldline)	Archive (Deep Archive)
Tần suất đọc	Hàng ngày	1 lần/tháng	1-2 lần/năm	Hiếm khi (vài năm/lần)
Độ trễ lấy dữ liệu	Tức thì (ms)	Tức thì (ms)	Vài phút đến vài giờ	3 đến 12 giờ
Phí lưu trữ (\$/GB)	Cao nhất	Trung bình	Rẻ	Siêu rẻ
Phí đọc/ghi dữ liệu	Rẻ / Miễn phí	Trung bình	Cao	Rất cao
Thời hạn lưu tối thiểu	Không quy định	30 ngày	90 ngày	180 ngày

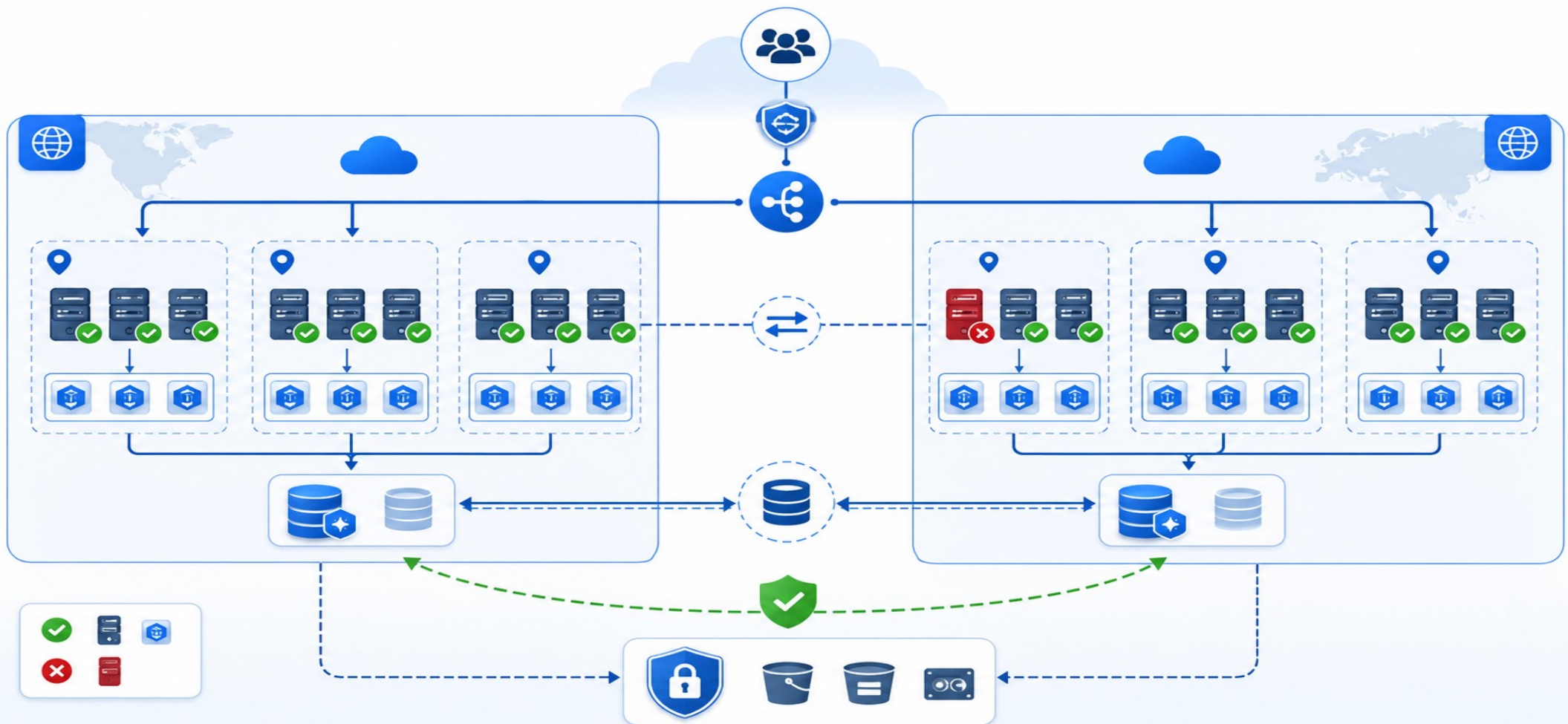
Sao lưu, snapshot và phục hồi

- Trong kiến trúc hạ tầng và lưu trữ đám mây, **Sao lưu (Backup)**, **Ảnh chụp nhanh (Snapshot)** và **Phục hồi (Recovery)** là 3 khái niệm cốt lõi thuộc chiến lược **Quản trị rủi ro & Đảm bảo tính liên tục của hoạt động kinh doanh (Business Continuity and Disaster Recovery - BCDR)**.
- Snapshot ghi lại trạng thái tại một thời điểm
- Backup có thể được lưu ở vùng hoặc tài khoản độc lập
- Phục hồi cần kiểm thử định kỳ, không chỉ cấu hình trên giấy
- Cần xác định RPO và RTO cho từng loại dữ liệu

Phân biệt Snapshot và Backup

Tiêu chí	Snapshot (Ảnh chụp nhanh)	Backup (Sao lưu)
Bản chất	Bản ghi chỉ số/con trỏ tại một thời điểm	Bản sao dữ liệu hoàn toàn độc lập
Vị trí lưu trữ	Thường nằm trên cùng hệ thống/mảng ổ đĩa	Lưu ở hạ tầng/vùng cách ly riêng (Object Storage/Region khác)
Tốc độ tạo	Cực nhanh (giây đến phút)	Lâu hơn (phụ thuộc vào dung lượng dữ liệu & băng thông)
Tốc độ phục hồi	Rất nhanh (chỉ cần khôi phục con trỏ/delta)	Lâu hơn (phải truyền tải toàn bộ dữ liệu về)
Mục đích chính	Phục hồi nhanh sự cố ngắn hạn (Lỗi Update, Dev/Test)	Lưu trữ dài hạn, chống thảm họa (Disaster Recovery), tuân thủ pháp lý

Sơ đồ kiến trúc phục hồi thảm họa trên đám mây



Lưu trữ bất biến

Lưu trữ bất biến (Immutable Storage) là cơ chế lưu trữ dữ liệu theo nguyên tắc **WORM (Write Once, Read Many - Ghi một lần, Đọc nhiều lần)**. Khi dữ liệu được ghi vào hệ thống lưu trữ bất biến, **không ai**—bao gồm cả quản trị viên hệ thống (System Admin), người dùng có đặc quyền cao nhất (Root/Global Admin), hay kẻ tấn công chiếm được tài khoản—có thể sửa đổi, ghi đè hoặc xóa bỏ dữ liệu đó trong một khoảng thời gian được thiết lập trước (Retention Period).

- Dữ liệu không thể bị sửa hoặc xóa trong thời gian lưu giữ
- Hữu ích cho tuân thủ, nhật ký kiểm toán và chống mã độc tổng tiền
- Cần thiết kế quyền quản trị cẩn thận
- Không thay thế hoàn toàn cho sao lưu và giám sát

Bảng So sánh: Lưu trữ thông thường vs. Lưu trữ bất biến

Tiêu chí	Lưu trữ thông thường (Standard Storage)	Lưu trữ bất biến (Immutable Storage)
Quyền Sửa / Xóa	Tài khoản Admin/Root có thể sửa hoặc xóa bất kỳ lúc nào	Không thể sửa/xóa trong thời hạn quy định (kể cả Root Admin)
Mức độ chống Ransomware	Thấp (Mã độc có thể chiếm quyền Admin để xóa bản sao lưu)	Cực cao (Mã độc không thể can thiệp vào bản sao lưu)
Tính toàn vẹn dữ liệu	Có rủi ro bị ghi đè, làm sai lệch dữ liệu	Đảm bảo dữ liệu nguyên bản 100% từ lúc tạo
Trường hợp sử dụng chính	Dữ liệu ứng dụng hàng ngày, dữ liệu cần cập nhật liên tục	Bản sao lưu (Backup), Nhật ký hệ thống (Audit Logs), Hồ sơ pháp lý

4. CƠ SỞ DỮ LIỆU QUAN HỆ TRÊN ĐÁM MÂY

- Cơ sở dữ liệu quan hệ trên đám mây (Cloud Relational Database / Managed Relational Database) là dịch vụ quản trị CSDL theo mô hình quan hệ (RDBMS) được triển khai, quản lý và vận hành hoàn toàn trên hạ tầng điện toán đám mây.
- Thay vì doanh nghiệp phải tự mua máy chủ vật lý, cài đặt hệ điều hành, cấu hình CSDL và tự thực hiện các thao tác bảo trì (sao lưu, nâng cấp, phân tầng lưu trữ, khôi phục...), các nhà cung cấp đám mây (Cloud Providers) sẽ đảm nhận toàn bộ phần hạ tầng bên dưới dưới dạng dịch vụ PaaS (Platform as a Service) hoặc DBaaS (Database as a Service).

Cơ sở dữ liệu tự quản trị và được quản lý

- Self-managed: người dùng cài đặt và vận hành CSDL trên máy ảo
- Managed database: nhà cung cấp quản lý hạ tầng, backup, patching, monitoring cơ bản
- Managed service giảm gánh nặng vận hành nhưng có giới hạn tùy biến
- Lựa chọn phụ thuộc kiểm soát, chi phí, kỹ năng và yêu cầu tuân thủ

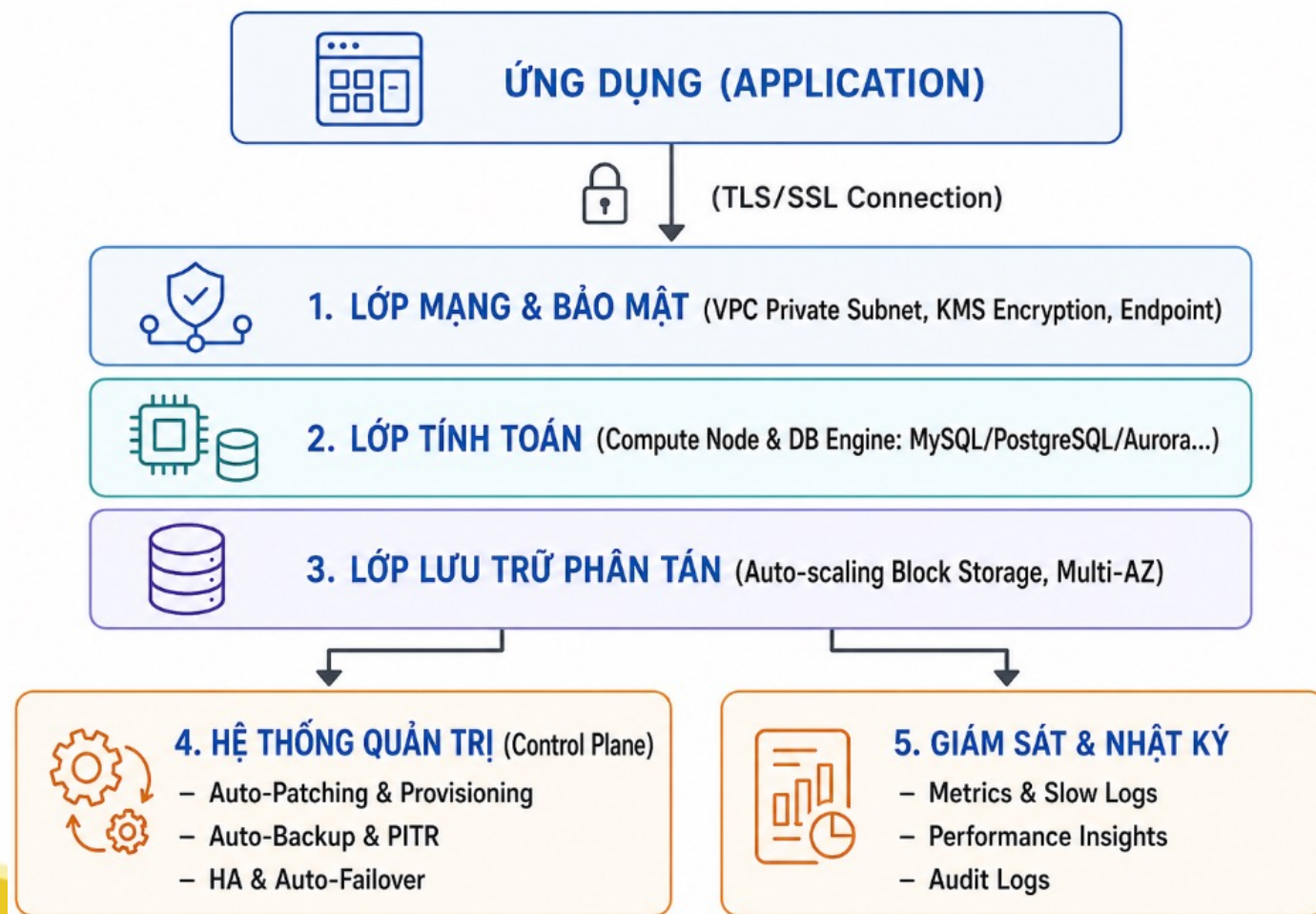
Bảng So sánh: CSDL Quan hệ Truyền thống (On-Premises) vs. Đám mây (DBaaS)

Tiêu chí	CSDL Truyền thống (On-Premises)	CSDL Quan hệ Đám mây (DBaaS)
Chi phí đầu tư	CapEx cao (mua phần cứng, license, phòng Server)	OpEx (trả theo mức độ sử dụng thực tế)
Bảo trì & Vá lỗi	Thủ công, đòi hỏi đội ngũ DBA chuyên trách	Tự động hóa 100% bởi nhà cung cấp đám mây
Mở rộng (Scaling)	Khó khăn, mất nhiều ngày/tuần để mua thêm phần cứng	Khởi tạo / Nâng cấp trong vài phút (hoặc Serverless)
Khả năng Chịu lỗi (HA)	Tốn kém, phức tạp khi cấu hình Clustering/Replication	Tích hợp sẵn qua vài click (Multi-AZ/Cross-Region)
Sao lưu & Phục hồi	Phải tự xây dựng kịch bản & quản lý hạ tầng Backup	Tự động tạo Snapshot, khôi phục theo mốc thời gian (PITR)

Khi nào nên chọn CSDL Quan hệ trên Đám mây?

- **Ứng dụng Tài chính, Ngân hàng, Thương mại điện tử:** Những hệ thống yêu cầu tính nhất quán dữ liệu tuyệt đối (ACID), các giao dịch phức tạp (Transactions) và tính sẵn sàng 24/7.
- **Doanh nghiệp muốn giảm tải vận hành:** Giúp đội ngũ phát triển (Developers) tập trung vào logic ứng dụng thay vì phải tốn thời gian quản trị máy chủ CSDL.
- **Hệ thống có tải biến động:** Cần khả năng mở rộng tài nguyên linh hoạt theo mùa cao điểm mà không lãng phí chi phí đầu tư ban đầu.

Các thành phần của cơ sở dữ liệu quan hệ được quản lý



Các thành phần của cơ sở dữ liệu quan hệ được quản lý

- **Một dịch vụ cơ sở dữ liệu quan hệ được quản lý (Managed Relational Database) (DBaaS) được thiết kế theo kiến trúc phân tầng (Layered Architecture) nhằm tách biệt giữa lớp tính toán, lớp lưu trữ và hệ thống quản trị tự động. Về cơ bản, một Managed Relational Database hoàn chỉnh trên đám mây (như AWS RDS, GCP Cloud SQL, Azure SQL DB) bao gồm 6 thành phần cốt lõi sau:**

1. Nút Tính toán & Engine CSDL (Compute Node & DB Engine)

Đây là "bộ não" xử lý mọi hoạt động của cơ sở dữ liệu:

- **DB Engine:** Nhận CSDL quan hệ xử lý cú pháp SQL, tối ưu hóa truy vấn (Query Optimizer), quản lý bộ nhớ đệm (Buffer Pool) và thực thi giao dịch (Transactions - ACID). Có thể là các engine truyền thống (MySQL, PostgreSQL, SQL Server, Oracle) hoặc các engine thuần đám mây (Aurora, Spanner).
- **Compute Instance (Virtual Machine/Container):** Tài nguyên CPU và RAM được cấp phát cho DB Engine.
- **Tách biệt Compute & Storage:** Trong các kiến trúc CSDL đám mây hiện đại (Cloud-Native), lớp tính toán hoàn toàn tách rời lớp lưu trữ, cho phép nâng cấp CPU/RAM (Vertical Scaling) hoặc tăng giảm số lượng nút đọc (Horizontal Scaling) mà không cần di chuyển dữ liệu.

2. Lớp Lưu trữ Phân tán (Distributed Storage Layer)

Nơi chứa toàn bộ dữ liệu (Data Files), nhật ký giao dịch (Redo/WAL Logs) và các bản chỉ mục (Indexes):

- **Block Storage / Network Attached Storage (NAS):** Dữ liệu không lưu trực tiếp trên ổ cứng cục bộ của máy chủ tính toán mà lưu trên hệ thống lưu trữ mạng có độ bền cao (High-Durability Storage).
- **Tự động mở rộng (Auto-scaling Storage):** Dung lượng đĩa tự động tăng khi dữ liệu chạm ngưỡng giới hạn mà không gây gián đoạn hệ thống.
- **Đồng bộ đa vùng (Multi-AZ Replication):** Ở tầng lưu trữ, dữ liệu thường được nhân bản (Replicate) đồng bộ hoặc bất đồng bộ sang nhiều ổ đĩa thuộc các Vùng sẵn sàng (Availability Zones - AZ) khác nhau để đảm bảo an toàn.

3. Hệ thống Tự động hóa & Quản trị (Control Plane / Management Service)

Thành phần phân biệt một Managed Database với CSDL tự cài đặt. Control Plane hoạt động như một "quản trị viên ảo" (Virtual DBA) đảm nhận các tác vụ:

- **Provisioning & Configuration:** Tự động khởi tạo, cấu hình tham số CSDL (Parameter Groups/Option Groups) chỉ qua API hoặc Giao diện người dùng (Console).
- **Auto-Patching:** Tự động cập nhật bản vá bảo mật và nâng cấp phiên bản hệ điều hành/DB Engine theo khung thời gian thiết lập (Maintenance Window).
- **Tự động Sao lưu (Automated Backup Engine):** Quản lý việc tạo Snapshot định kỳ và lưu trữ nhật ký giao dịch (Transaction Logs) phục vụ tính năng phục hồi theo mốc thời gian (Point-in-Time Recovery - PITR).
- **Quản lý Vòng đời Dữ liệu:** Tự động kết nối và đẩy các bản Backup cũ sang các tầng lưu trữ giá rẻ (Warm/Cold Storage) hoặc áp dụng lưu trữ bất biến (**Immutable Storage / Object Lock**) để chống Ransomware.

4. Cơ chế High Availability (HA) & Failover

Thành phần đảm bảo tính liên tục của hệ thống (Business Continuity):

- **Primary (Writer) & Standby (Reader) Instance:** Dữ liệu từ nút chính (Primary) được nhân bản liên tục sang nút dự phòng (Standby) ở một AZ khác.
- **Health Check & Auto-Failover Agent:** Hệ thống giám sát liên tục trạng thái hoạt động (Health status) của nút chính. Khi phát hiện sự cố phần cứng hoặc mất kết nối:
 - ✓ Tự động chuyển đổi vai trò của nút Standby thành Primary.
 - ✓ Tự động cập nhật địa chỉ IP/DNS Endpoint để ứng dụng tiếp tục truy cập mà không cần can thiệp thủ công.

5. Mạng & Lớp Bảo mật (Networking & Security Layer)

Cung cấp môi trường cô lập và bảo vệ dữ liệu ở mức tối đa:

- **VPC/VNet Isolation:** CSDL luôn được đặt trong mạng nội bộ riêng tư (Private Subnet), hoàn toàn ẩn khỏi Internet.
- **Endpoint / Connection Management:** Cung cấp đường dẫn kết nối (DNS Endpoint) duy nhất cho ứng dụng, giúp che giấu hạ tầng IP thực sự bên dưới.
- **Mã hóa (Encryption):**
 - **At-Rest (Mã hóa dữ liệu nghỉ):** Sử dụng các khóa KMS (Key Management Service) để mã hóa dữ liệu trên ổ đĩa, bản sao lưu và Snapshot.
 - **In-Transit (Mã hóa dữ liệu di chuyển):** Bắt buộc mọi kết nối từ ứng dụng đến CSDL phải qua giao thức mã hóa TLS/SSL.
- **Quản lý Truy cập (IAM & Database Auth):** Tích hợp phân quyền theo vai trò (RBAC), xác thực qua IAM Token hoặc Active Directory/LDAP.

6. Hệ thống Giám sát, Cảnh báo & Nhật ký (Monitoring, Observability & Logging)

Cung cấp cái nhìn toàn diện về sức khỏe và hiệu năng của CSDL:

- **Metrics & Dashboards:** Theo dõi thời gian thực các chỉ số quan trọng như CPU Utilization, Memory, IOPS, Storage Usage, Connection Count.
- **Performance Insights:** Phân tích sâu các câu lệnh SQL chạy chậm (Slow Queries), các tiến trình gây tắc nghẽn (Lock/Deadlock) để giúp tối ưu hóa hệ thống.
- **Audit & Error Logs:** Tự động ghi lại nhật ký lỗi hệ thống, nhật ký truy cập (Audit Logs) phục vụ công tác kiểm toán và tuân thủ pháp lý.

CSDL phân tán toàn cầu

- **Cơ sở dữ liệu phân tán toàn cầu (Global Distributed Database / Globally Distributed Database)** là hệ thống CSDL có dữ liệu được lưu trữ, đồng bộ và quản trị trên nhiều nút (nodes) nằm ở nhiều trung tâm dữ liệu (Data Centers) hoặc các vùng địa lý khác nhau (Multi-Region / Cross-Region) trên phạm vi toàn cầu.
- Khác với CSDL phân tán truyền thống (chỉ chạy trong một Data Center hoặc 1 Region), CSDL phân tán toàn cầu phải giải quyết bài toán cực kỳ phức tạp về **độ trễ mạng do khoảng cách địa lý (Network Latency)**, **tính nhất quán dữ liệu (Data Consistency)** và **khả năng chịu lỗi khi toàn bộ một Region bị mất kết nối (Region Failure)**.

Bảng so sánh nhanh các CSDL phân tán toàn cầu tiêu biểu

CSDL	Loại hình	Cơ chế đồng bộ / Đồng thuận	Mức độ nhất quán	Điểm nổi bật
Google Spanner	Distributed SQL	Paxos + TrueTime API (GPS/Atomic Clock)	Strong Consistency (ACID)	Tiêu chuẩn vàng cho giao dịch tài chính toàn cầu
CockroachDB	Distributed SQL	Raft Consensus	Strong Consistency (ACID)	Mã nguồn mở, tương thích PostgreSQL, Geo-partitioning tốt
AWS DynamoDB Global Tables	NoSQL (Key-Value)	Multi-Region Active-Active Replication	Eventual / Strong (Cục bộ)	Đơn giản, hoàn toàn Serverless, độ trễ cực thấp
Azure Cosmos DB	Multi-Model NoSQL	Multi-Master Replication	Configurable (5 mức độ)	Cung cấp SLA khắt khe về độ trễ, khả năng sẵn sàng 99.999%
Apache Cassandra	NoSQL (Wide-Column)	Peer-to-Peer (Gossip Protocol)	Eventual Consistency	Tốc độ ghi cực cao, không có điểm lỗi đơn lẻ (No Single Point of Failure)

5. CƠ SỞ DỮ LIỆU NoSQL



Vì sao cần NoSQL?

- Dữ liệu ngày càng đa dạng và thay đổi nhanh
- Nhu cầu mở rộng ngang quy mô lớn
- Một số bài toán không phù hợp mô hình bảng quan hệ
- NoSQL đánh đổi một phần mô hình quan hệ để đạt linh hoạt, tốc độ hoặc quy mô
- **Lưu ý:** NoSQL không ra đời để thay thế hoàn toàn SQL, mà để bổ sung cho SQL. Trong các hệ thống lớn hiện đại, mô hình **Polyglot Persistence** (sử dụng song song cả SQL cho giao dịch thanh toán và NoSQL cho caching/feed/log) là kiến trúc chuẩn mực được áp dụng rộng rãi.

So sánh Động lực Chuyển đổi

Tiêu chí	CSDL Quan hệ (SQL)	CSDL NoSQL
Mô hình Mở rộng	Scale-Up (Thêm CPU/RAM vào 1 máy chủ)	Scale-Out (Thêm nhiều máy chủ vào cụm)
Cấu trúc Dữ liệu	Schema cố định, bảng & hàng	Linh hoạt (JSON, Key-Value, Graph, Column)
Phép toán Phức tạp	Tối ưu cho JOIN, Aggregation, ACID	Tối ưu cho Đọc/Ghi nhanh, phi chuẩn hóa
Mức độ Nhất quán	Strong Consistency (Mọi lúc)	Eventual Consistency (Ưu tiên Sẵn sàng)
Trường hợp Sử dụng	Giao dịch Tài chính, Ngân hàng, ERP	Big Data, Real-time Analytics, IoT, MXH, Caching

Các nhóm NoSQL phổ biến

- Key-value: truy xuất rất nhanh theo khóa
- Document: lưu tài liệu JSON/BSON linh hoạt
- Column-family: xử lý dữ liệu lớn theo cột hoặc họ cột
- Graph: biểu diễn quan hệ phức tạp giữa thực thể
- Time-series: dữ liệu theo thời gian
- Vector: tìm kiếm tương đồng cho AI/GenAI

Các nhóm NoSQL phổ biến

- Key-value: truy xuất rất nhanh theo khóa
- Document: lưu tài liệu JSON/BSON linh hoạt
- Column-family: xử lý dữ liệu lớn theo cột hoặc họ cột
- Graph: biểu diễn quan hệ phức tạp giữa thực thể
- Time-series: dữ liệu theo thời gian
- Vector: tìm kiếm tương đồng cho AI/GenAI

1. Key-Value Stores (CSDL Dạng Khóa - Giá trị)

Mô hình đơn giản nhất trong các loại NoSQL. Dữ liệu được lưu trữ dưới dạng một bảng băm (Hash Table) khổng lồ, trong đó mỗi chuỗi thông tin gồm một Key (Khóa) duy nhất và một Value (Giá trị) tương ứng.

- **Đặc điểm cốt lõi: Tốc độ đọc/ghi siêu nhanh:** Do truy xuất thẳng qua Key với độ phức tạp $O(1)$.
 - ✓ **Value linh hoạt:** Giá trị có thể là bất kỳ thứ gì, từ chuỗi đơn giản, số, JSON cho đến tệp nhị phân (Binary Data).
 - ✓ **Hạn chế truy vấn:** Không hỗ trợ truy vấn theo thuộc tính bên trong Value (muốn lấy dữ liệu phải biết chính xác Key).
- **Trường hợp sử dụng:** Bộ nhớ đệm (Caching - Redis/Memcached), Quản lý phiên làm việc người dùng (Session State), Giỏ hàng trực tuyến (Shopping Carts), Hệ thống xếp hạng (Leaderboards).
- **Đại diện tiêu biểu:** Redis, Amazon DynamoDB, Memcached, Riak.

1. Document Stores (CSDL Dạng Tài liệu)

Dữ liệu được lưu trữ dưới dạng các tài liệu (Document) bán cấu trúc, phổ biến nhất là **JSON, BSON, hoặc XML**. Mỗi tài liệu chứa cặp key-value và có thể chứa các tài liệu con lồng nhau (nested documents) hoặc mảng (arrays).

- **Đặc điểm cốt lõi:**
 - ✓ **Lược đồ linh hoạt (Schema-less):** Các tài liệu trong cùng một bộ sưu tập (Collection) không bắt buộc phải có cùng cấu trúc.
 - ✓ **Truy vấn tự nhiên:** Dễ dàng ánh xạ trực tiếp với các đối tượng (Objects) trong mã nguồn ứng dụng (OOP).
- **Trường hợp sử dụng:** Hệ thống quản lý nội dung (CMS), Website Thương mại điện tử (khi danh mục sản phẩm có thuộc tính thay đổi liên tục), Hồ sơ người dùng (User Profiles), Catalogs.
- **Đại diện tiêu biểu:** MongoDB, Couchbase, Amazon DocumentDB.

3. Wide-Column / Column-Family Stores (CSDL Dạng Cột Rộng)

Thay vì lưu trữ dữ liệu theo từng hàng (Row-oriented) như RDBMS, CSDL dạng cột lưu trữ dữ liệu theo từng **nhóm cột (Column Families)** trên đĩa cứng.

- **Đặc điểm cốt lõi:**
 - **Khả năng mở rộng cực lớn:** Thiết kế chuyên dụng để hoạt động trên hàng nghìn máy chủ phân tán (Scale-Out).
 - **Tối ưu hóa đọc/ghi hàng loạt:** Đọc/ghi một lượng cực lớn dữ liệu trên một vài cột cụ thể rất nhanh mà không cần quét qua toàn bộ hàng.
 - **Thích hợp cho Big Data:** Xử lý hiệu quả dữ liệu quy mô hàng Petabyte.
- **Trường hợp sử dụng:** Lưu trữ dữ liệu chuỗi thời gian (Time-series data), Nhật ký hệ thống (System Logs), Dữ liệu từ thiết bị IoT, Phân tích dữ liệu hành vi người dùng, Ứng dụng tin nhắn nhắn tin lớn (Messenger/WhatsApp).
- **Đại diện tiêu biểu:** Apache Cassandra, HBase, ScyllaDB, Google Cloud Bigtable.

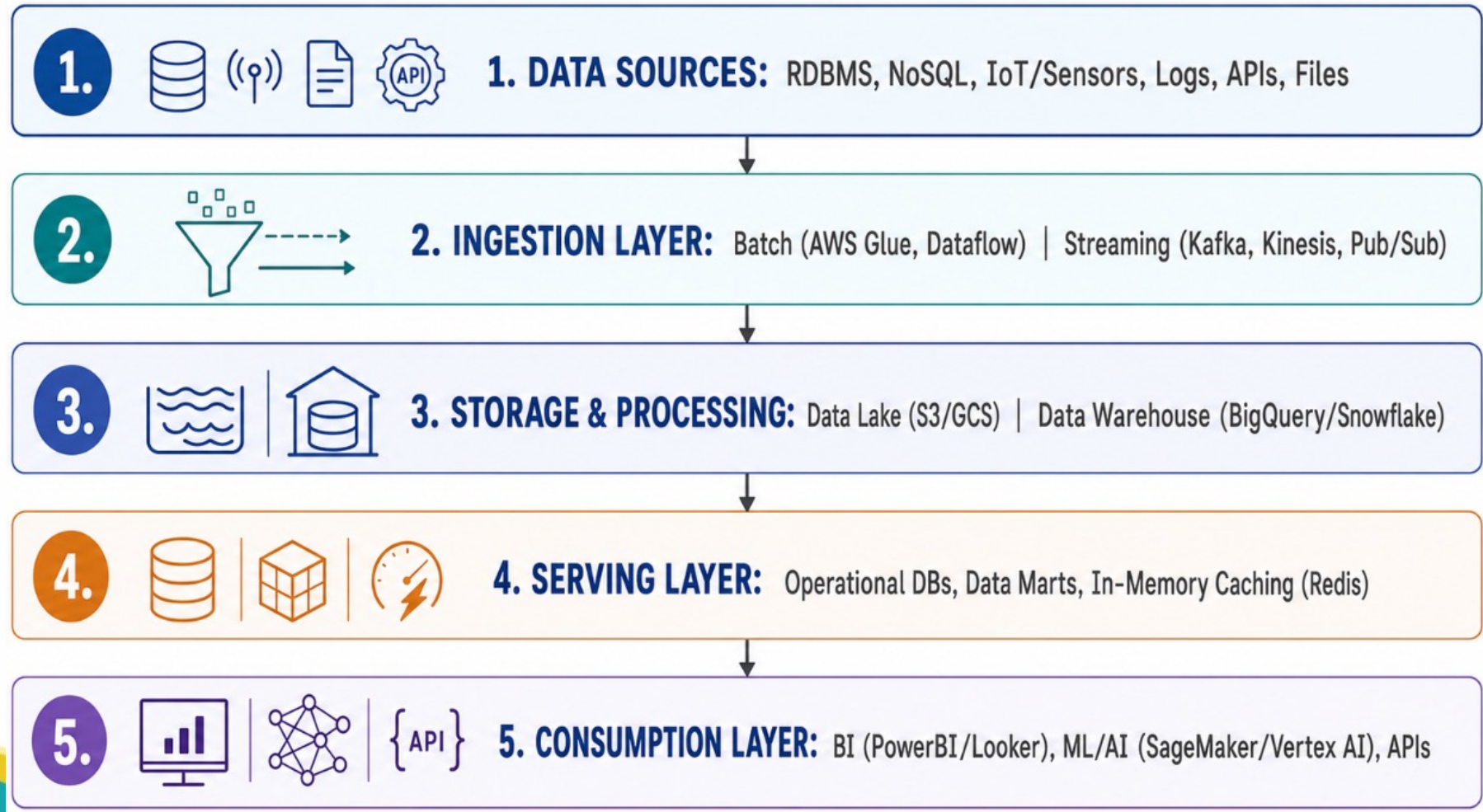
4. Graph Databases (CSDL Dạng Đồ thị)

Sử dụng lý thuyết đồ thị để lưu trữ dữ liệu, bao gồm các **Nút (Nodes - đối tượng)**, **Cạnh (Edges - mối quan hệ)** và **Thuộc tính (Properties)** của chúng.

- **Đặc điểm cốt lõi:**
 - ✓ Tối ưu cho mối quan hệ phức tạp: Việc truy vấn mối quan hệ nhiều-nhiều (Many-to-Many) cực kỳ nhanh chóng mà không cần phép JOIN tốn kém như trong SQL.
 - ✓ Truy vết chiều sâu: Cho phép tìm kiếm các đường đi (Paths) hoặc mối quan hệ ẩn giữa các đối tượng qua nhiều tầng trung gian.
- **Trường hợp sử dụng:** Mạng xã hội (Friend Recommendations, Follows), Hệ thống phát hiện gian lận tài chính/ngân hàng (Fraud Detection), Engine gợi ý sản phẩm (Recommendation Engines), Đồ thị trí thức (Knowledge Graphs).
- **Đại diện tiêu biểu:** Neo4j, Amazon Neptune, ArangoDB, JanusGraph.

Nhóm NoSQL	Cấu trúc dữ liệu chính	Ưu điểm vượt trội	Nhược điểm lớn nhất	Đại diện phổ biến
Document	JSON / BSON	Linh hoạt, tự nhiên với Dev, dễ truy vấn	Dung lượng tốn hơn do lặp Key	MongoDB, Couchbase
Key-Value	Cặp Key - Value	Độ trễ cực thấp, tốc độ ghi/đọc siêu nhanh	Không truy vấn phức tạp được	Redis, DynamoDB
Wide-Column	Nhóm các Cột (Column Family)	Mở rộng ngang vô hạn, ghi Big Data mượt	Khó truy vấn nếu không thiết kế đúng Key	Cassandra, HBase
Graph	Nodes, Edges, Properties	Xử lý mối quan hệ siêu linh hoạt	Khó mở rộng ngang (Sharding) hơn các loại khác	Neo4j, Neptune

6. KIẾN TRÚC DỮ LIỆU TRÊN ĐÁM MÂY



Các Mô hình Kiến trúc Dữ liệu Đám mây Phổ biến

Tùy thuộc vào nhu cầu về tốc độ, chi phí và quy mô, doanh nghiệp thường chọn một hoặc kết hợp các mô hình kiến trúc sau:

- Data Warehouse trên Đám mây (Modern Cloud Data Warehouse)
- Data Lake (Hồ dữ liệu)
- Data Lakehouse (Kiến trúc Lai Hiện đại)
- Kiến trúc Lambda & Kappa (Xử lý Real-time & Batch)

Data Warehouse trên Đám mây (Modern Cloud Data Warehouse)

- **Khái niệm:** Hệ thống lưu trữ tập trung dữ liệu **đã có cấu trúc (Structured Data)**, tối ưu cho các truy vấn phân tích phức tạp (OLAP).
- **Đặc điểm:** Tách biệt giữa Compute (Tính toán) và Storage (Lưu trữ) giúp tối ưu chi phí.
- **Đại diện:** Google BigQuery, Snowflake, AWS Redshift, Azure Synapse Analytics.

Data Lake (Hồ dữ liệu)

- **Khái niệm:** Nơi lưu trữ **mọi dạng dữ liệu** (cấu trúc, bán cấu trúc JSON/XML, phi cấu trúc như video, hình ảnh, audio) ở dạng thô nguyên bản với chi phí cực rẻ.
- **Đặc điểm:** Sử dụng các Object Storage (S3, GCS, ADLS Gen2). Dữ liệu chỉ được định dạng cấu trúc khi truy vấn (**Schema-on-Read**).

Data Lakehouse (Kiến trúc Lai Hiện đại)

- **Khái niệm:** Xu hướng kiến trúc hợp nhất những ưu điểm tuyệt vời nhất của cả **Data Lake** (chi phí rẻ, chứa mọi dạng dữ liệu) và **Data Warehouse** (quản trị ACID, hiệu năng truy vấn SQL cực nhanh).
- **Công nghệ cốt lõi:** Các định dạng bảng mã nguồn mở như **Apache Iceberg**, **Delta Lake**, **Apache Hudi**.
- **Đại diện tiêu biểu:** Databricks Lakehouse Platform.

Kiến trúc Lambda & Kappa (Xử lý Real-time & Batch)

- **Lambda Architecture:** Chia làm 2 luồng riêng biệt: **Batch Layer** (xử lý chính xác dữ liệu lịch sử) và **Speed Layer** (xử lý dữ liệu thời gian thực với độ trễ thấp).
- **Kappa Architecture:** Đơn giản hóa Lambda bằng cách loại bỏ luồng Batch, coi **mọi dữ liệu là một chuỗi Stream liên tục** và xử lý qua Stream Processing Engine (như Apache Flink, Spark Streaming).

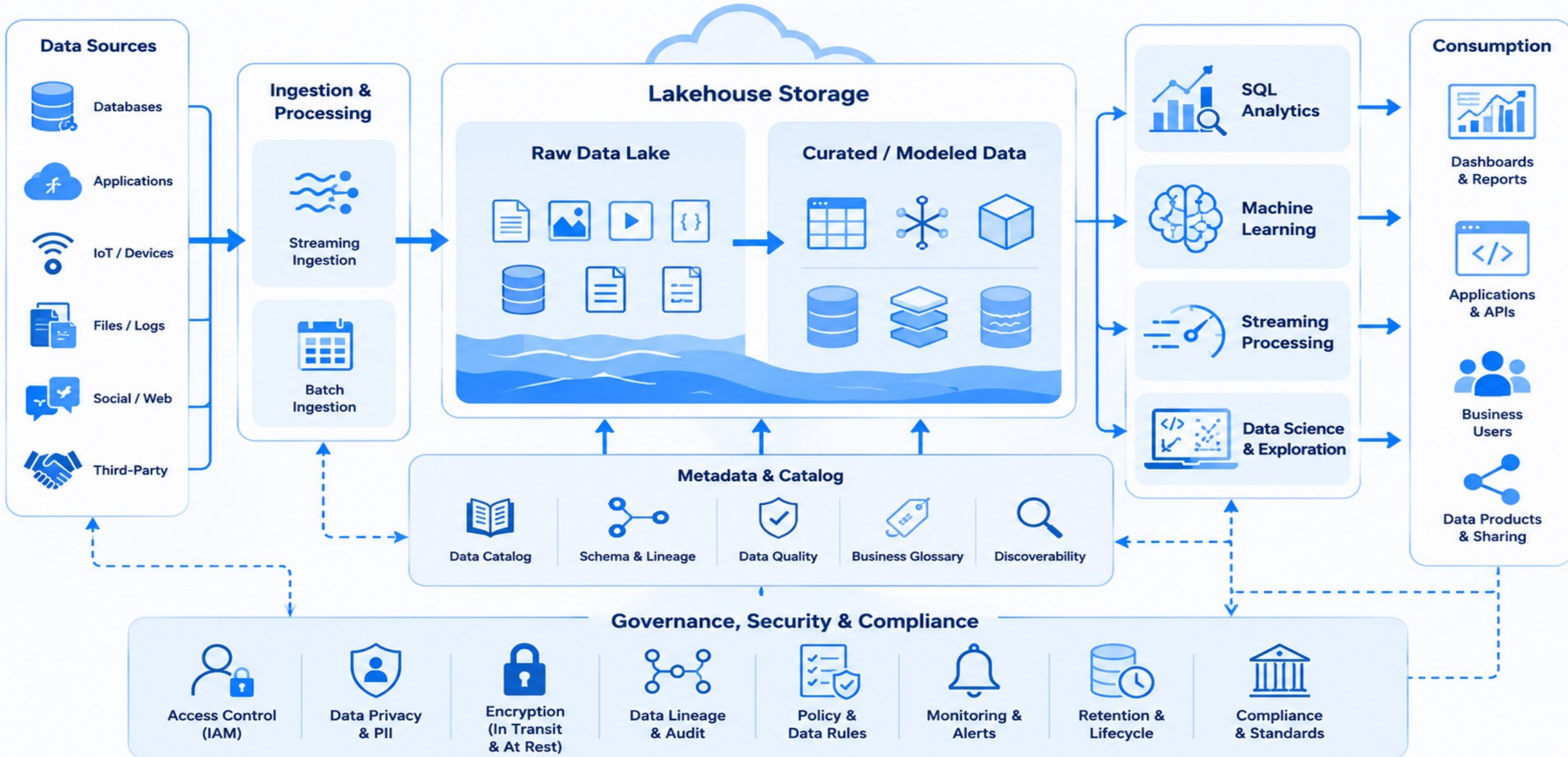
Các Nguyên tắc Thiết kế Kiến trúc Dữ liệu Đám mây Chuẩn mực

Để xây dựng một kiến trúc bền vững, chuyên nghiệp cần tuân thủ các nguyên tắc vàng sau:

- **Tách biệt Compute và Storage (Decoupling Compute & Storage):** Cho phép mở rộng tài nguyên tính toán độc lập với dung lượng đĩa lưu trữ, giúp tiết kiệm tối đa chi phí.
- **Tự động hóa & Serverless:** Ưu tiên sử dụng các dịch vụ không máy chủ (Serverless Data Pipelines) để giảm thiểu gánh nặng vận hành và tự động mở rộng theo tải.
- **Định dạng dữ liệu tối ưu (Columnar File Formats):** Lưu trữ dữ liệu phân tích ở các định dạng cột nén cao như **Parquet** hoặc **ORC** thay vì CSV/JSON để tăng tốc độ truy vấn và giảm chi phí IOPS.
- **Quản trị & An toàn Dữ liệu (Data Governance & Security):**
 - **Mã hóa (Encryption):** Mã hóa toàn bộ dữ liệu ở trạng thái nghỉ (At-Rest) và di chuyển (In-Transit).
 - **Phân quyền dòng/cột (Fine-grained Access Control):** Kiểm soát chính xác ai được xem dữ liệu gì dựa trên RBAC/ABAC.
 - **Data Lineage:** Quản lý vòng đời và nguồn gốc dòng chảy dữ liệu.
- **Mô hình hóa dữ liệu linh hoạt (Data Mesh / Data Fabric):** Với các tập đoàn lớn, chuyển từ kiến trúc tập trung sang **Data Mesh** — chia dữ liệu thành các miền sở hữu riêng (Data Domains) do từng đội nhóm kinh doanh tự quản lý và cung cấp dưới dạng sản phẩm (Data as a Service).

Tiêu chí Lựa chọn Công nghệ

Tiêu chí	Data Lake	Cloud Data Warehouse	Data Lakehouse
Loại Dữ liệu	Phi cấu trúc, Bán cấu trúc, Cấu trúc	Chủ yếu là Cấu trúc	Mọi loại dữ liệu
Chi phí Lưu trữ	Cực kỳ Rẻ (Object Storage)	Trung bình - Cao	Rẻ (Object Storage nền)
Đối tượng Phục vụ	Data Scientist, AI/ML Engineers	BI Analyst, Data Analyst	Cả Data Engineer, Analyst & AI/ML
Tính chất Giao dịch	Không hỗ trợ/Hỗ trợ kém	Hỗ trợ đầy đủ ACID	Hỗ trợ chuẩn ACID (Delta/Iceberg)
Hiệu năng SQL	Trung bình	Cực nhanh	Rất nhanh

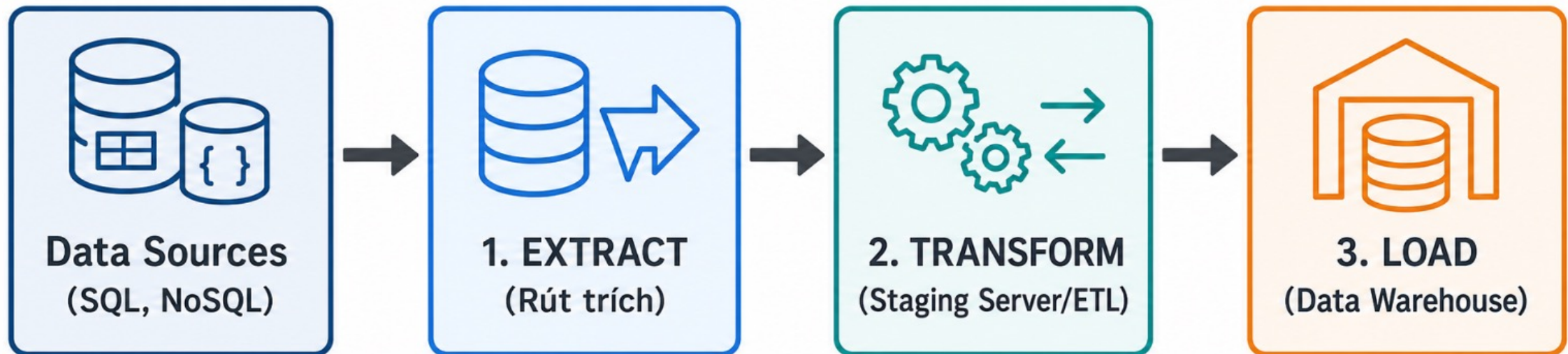


ETL và ELT

- **ETL (Extract - Transform - Load) và ELT (Extract - Load - Transform)** là hai quy trình tích hợp dữ liệu (Data Integration Pipelines) cốt lõi nhằm thu thập dữ liệu từ nhiều nguồn khác nhau, xử lý và đưa vào hệ thống lưu trữ tập trung để phục vụ phân tích (BI, Reporting, AI/ML).
- Sự khác biệt căn bản giữa hai mô hình này nằm ở **thứ tự thực hiện bước Biến đổi dữ liệu (Transform) và nơi diễn ra quá trình tính toán đó.**

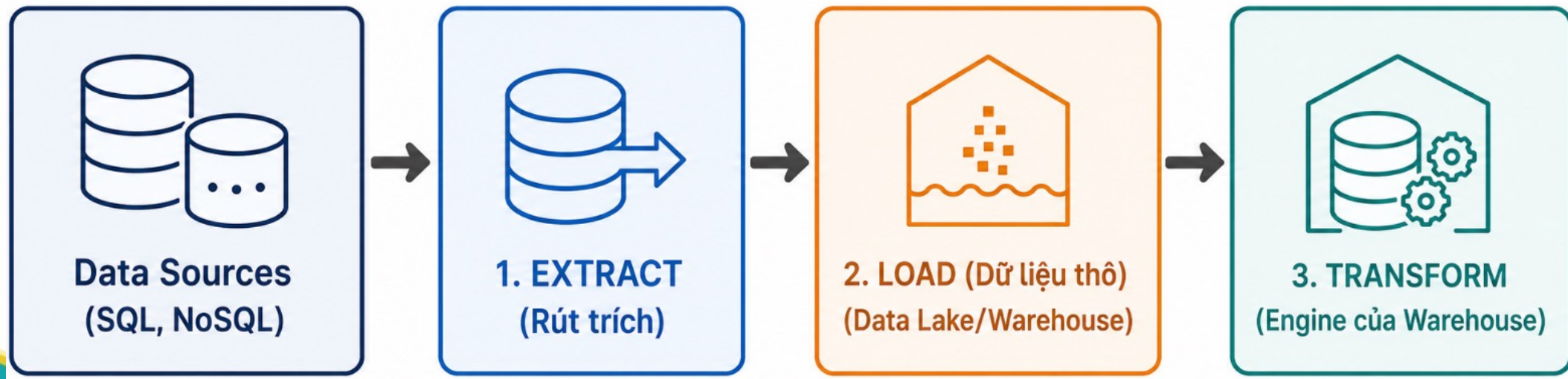
ETL (Extract - Transform - Load) — Mô hình Truyền thống

Trong quy trình ETL, dữ liệu được biến đổi **trước khi** đưa vào hệ thống lưu trữ đích.



ELT (Extract - Load - Transform) — Mô hình Đám mây Hiện đại

- Với sự bùng nổ của hạ tầng Đám mây (Cloud) và các hệ thống lưu trữ/tính toán phân tán, mô hình ELT ra đời bằng việc đảo thứ tự: **Tải dữ liệu thô vào trước, biến đổi sau.**

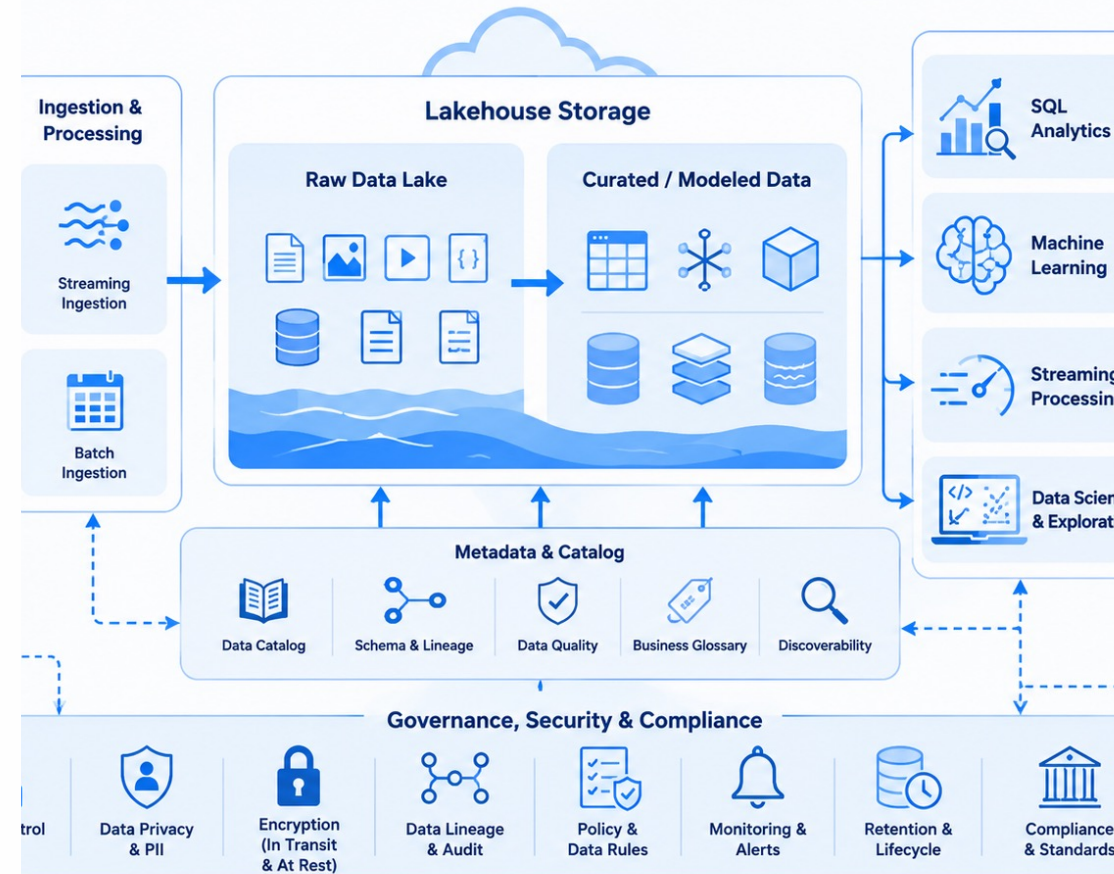


So sánh Chi tiết ETL vs ELT

Tiêu chí	ETL (Extract - Transform - Load)	ELT (Extract - Load - Transform)
Môi trường phù hợp	Hệ thống On-Premise (Tại chỗ), Data Warehouse truyền thống	Cloud Data Warehouse (Snowflake, BigQuery), Data Lakehouse
Nơi xử lý Transform	Máy chủ trung gian (Staging/ETL Server)	Ngay trên CSDL Đích (Target Database/Engine)
Loại dữ liệu	Chủ yếu là Dữ liệu có cấu trúc (Structured)	Cấu trúc, Bán cấu trúc (JSON) & Phi cấu trúc
Tốc độ nạp dữ liệu	Chậm (Chờ biến đổi xong mới nạp)	Cực nhanh (Nạp trực tiếp dữ liệu thô)
Tốc độ biến đổi	Phụ thuộc vào máy chủ ETL	Rất nhanh (Tận dụng Massively Parallel Processing - MPP)
Khả năng bảo mật	Dễ tuân thủ (Mã hóa/Lọc PII trước khi nạp)	Cần thiết lập phân quyền/mã hóa kỹ trên Data Lake
Công nghệ tiêu biểu	Informatica, Talend, IBM DataStage, SSIS	dbt (data build tool), AWS Glue, Airflow, Databricks

Metadata và Data Catalog

- Trong kiến trúc dữ liệu hiện đại (đặc biệt khi xử lý quy mô Big Data trên Cloud), nếu dữ liệu là **"tài sản"** thì Metadata chính là **"bản đồ/nhãn mác"** còn Data Catalog chính là **"thư viện trung tâm"** giúp doanh nghiệp tìm kiếm, quản lý và khai thác tài sản đó một cách tối ưu.



1. Metadata (Siêu dữ liệu) — "Dữ liệu về Dữ liệu"

- **Metadata** là những thông tin ngữ cảnh mô tả chi tiết về đặc tính, nguồn gốc, cấu trúc và chất lượng của dữ liệu chính (Data).
- Nếu không có Metadata, dữ liệu thô sẽ giống như một cuốn sách không có bìa, không có chỉ mục và không có tên tác giả — người dùng không thể biết nó dùng để làm gì hay có đáng tin cậy hay không.

2. Data Catalog (Danh mục Dữ liệu) — "Thư viện Dữ liệu Doanh nghiệp"

- **Data Catalog** là một phần mềm/dịch vụ trung tâm có chức năng thu thập, sắp xếp, chỉ mục (indexing) và quản lý toàn bộ Metadata đến từ tất cả các nguồn dữ liệu trong doanh nghiệp (RDBMS, NoSQL, Data Lake, Data Warehouse, BI Tools).
- Data Catalog đóng vai trò như một **máy tìm kiếm Google nội bộ** dành cho dữ liệu của doanh nghiệp.

Mối quan hệ giữa Metadata và Data Catalog

Mối quan hệ giữa hai khái niệm này có thể ví như **Nhiên liệu** và **Động cơ**:

- **Metadata là Nguyên liệu cốt lõi:** Data Catalog không tự tạo ra giá trị nếu không có Metadata. Catalog sẽ liên tục quét (scan/crawl) các hệ thống CSDL để rút trích Metadata.
- **Data Catalog là Nền tảng quản lý:** Data Catalog thu gom các luồng Metadata rời rạc, tổ chức lại thành một giao diện trực quan và hữu ích cho người dùng cuối.
- **Data Catalog=Automated Metadata Collection+Search Engine+Data Lineage+Governance UI**

Vì sao Metadata & Data Catalog trở thành bắt buộc trong Đám mây?

- **Giải quyết bài toán "Data Swamp" (Đầm lầy dữ liệu):** Khi doanh nghiệp xây dựng Data Lake trên Cloud (như S3, GCS), họ dễ dàng đổ hàng Petabyte dữ liệu thô vào đó. Nếu không có Data Catalog để gắn nhãn Metadata, Data Lake sẽ nhanh chóng trở thành một "đầm lầy" không ai biết dữ liệu nào nằm ở đâu.
- **Thúc đẩy Mô hình Self-Service Analytics:** Tự phục vụ dữ liệu. Các nhà phân tích kinh doanh (BA/DA) có thể tự truy tìm và hiểu dữ liệu thông qua Data Catalog mà không cần phải liên tục tạo ticket nhờ Data Engineer giải thích hay lấy hộ dữ liệu.
- **Tuân thủ Pháp lý & An toàn An ninh mạng:** Hỗ trợ tuân thủ các quy định bảo vệ dữ liệu nghiêm ngặt (như Luật An ninh mạng Việt Nam, GDPR) nhờ khả năng định vị chính xác mọi nơi đang lưu trữ dữ liệu cá nhân (PII) thông qua việc tự động quét Metadata.

Các Công nghệ Data Catalog Tiêu biểu

- **Dịch vụ Cloud Native (Tích hợp sẵn):**
 - **AWS:** AWS Glue Data Catalog
 - **Google Cloud:** Dataplex (trước đây là Google Cloud Data Catalog)
 - **Microsoft Azure:** Microsoft Purview
- **Nền tảng Quản trị Dữ liệu Doanh nghiệp (Enterprise Data Catalog):**
 - **Mã nguồn mở:** Apache Atlas, OpenMetadata, DataHub (do LinkedIn phát triển)
 - **Thương mại/Enterprise:** Alation, Collibra, Informatica Enterprise Data Catalog

Quản trị dữ liệu trên cloud

- **Quản trị Dữ liệu trên Đám mây (Cloud Data Governance)** là tập hợp các nguyên tắc, chính sách, quy trình và công nghệ nhằm đảm bảo dữ liệu trên hạ tầng Cloud luôn **đáng tin cậy, an toàn, riêng tư, tuân thủ pháp luật và tối ưu về chi phí.**
- Nếu Kiến trúc Dữ liệu (Data Architecture) đóng vai trò xây dựng "đường ống và kho chứa", thì Quản trị Dữ liệu chính là "luật giao thông và lực lượng kiểm soát" giúp toàn bộ hệ thống vận hành đúng chuẩn và an toàn.

Các Trụ cột Cốt lõi của Cloud Data Governance

CLOUD DATA GOVERNANCE



Data Quality
(Chất lượng)



**Security &
Access Control**



Compliance
(Tuân thủ)



**Lifecycle
Management**



**Metadata &
Lineage**



Quản trị Chi phí Cloud
(Data FinOps)

Các Nguyên tắc Vàng cho Cloud Data Governance thành công

- **Tự động hóa tối đa (Automation First):** Không dùng quy trình thủ công để gán nhãn hay duyệt quyền. Hãy sử dụng các công cụ AI/ML để quét tự động (Scan) và phân loại dữ liệu nhạy cảm ngay khi dữ liệu vừa nạp vào Cloud.
- **Quản trị dưới dạng Mã (Governance as Code):** Nhúng các chính sách an toàn, phân quyền trực tiếp vào hạ tầng mã nguồn (Terraform, CI/CD pipelines) thay vì cấu hình bằng tay trên giao diện Cloud.
- **Bắt đầu nhỏ, Mở rộng nhanh (Start Small, Scale Fast):** Bắt đầu bằng việc quản trị một tập dữ liệu quan trọng nhất (như Dữ liệu Khách hàng/Giao dịch tài chính) để chứng minh giá trị, sau đó mới nhân rộng ra toàn bộ Data Lake/Warehouse.
- **Cân bằng giữa Bảo mật và Tốc độ (Security vs. Agility):** Quản trị dữ liệu không phải là "thắt chặt đến mức không ai dùng được", mà là tạo ra một môi trường an toàn để mọi người **tự tin sử dụng dữ liệu** phục vụ đổi mới sáng tạo (Self-Service Analytics).

Chất lượng dữ liệu CLOUD

- **Chất lượng Dữ liệu Đám mây (Cloud Data Quality)** là mức độ đáp ứng của dữ liệu trên hạ tầng đám mây về tính chính xác, nhất quán và độ tin cậy nhằm phục vụ cho các bài toán phân tích kinh doanh (BI), vận hành và ứng dụng Mô hình Trí tuệ Nhân tạo / Học máy (AI/ML).
- Trong môi trường Cloud với đặc thù thu thập dữ liệu đa nguồn và tốc độ cao (Streaming/Batch), chất lượng dữ liệu được ví như "**nguồn nước sạch**" — nếu đầu vào bị ô nhiễm (Trash-in), toàn bộ báo cáo và mô hình AI đầu ra sẽ cho kết quả sai lệch (Trash-out).
- **6 Tiêu chí (Dimensions) Đánh giá Chất lượng Dữ liệu**

6 Tiêu chí (Dimensions) Đánh giá Chất lượng Dữ liệu

- **Tính Chính xác (Accuracy):** Dữ liệu phải phản ánh đúng thực tế khách quan (ví dụ: Địa chỉ khách hàng, giá trị đơn hàng phải đúng với thực tế giao dịch).
- **Tính Đầy đủ (Completeness):** Không bị khuyết thiếu các trường dữ liệu quan trọng (ví dụ: Bảng thông tin người dùng không được bỏ trống các trường bắt buộc như **Email** hoặc **Mã số thuế**).
- **Tính Nhất quán (Consistency):** Dữ liệu đồng nhất trên toàn bộ hệ thống ngay cả khi nằm ở các kho chứa khác nhau (ví dụ: Định dạng ngày tháng phải chuẩn hóa YYYY-MM-DD ở tất cả các dịch vụ, không thể nơi dùng DD/MM/YYYY nơi dùng MM/DD/YYYY).
- **Tính Kịp thời (Timeliness / Freshness):** Dữ liệu phản ánh trạng thái mới nhất và sẵn sàng cho truy vấn đúng thời điểm yêu cầu (ví dụ: Dữ liệu biến động giá chứng khoán phải cập nhật theo thời gian thực với độ trễ tính bằng millisecond).
- **Tính Duy nhất (Uniqueness / Non-duplication):** Mỗi đối tượng thực thể chỉ tồn tại một bản ghi duy nhất, không bị trùng lặp bản ghi (Duplicate Records).
- **Tính Hợp lệ (Validity):** Dữ liệu phải tuân thủ đúng định dạng, cấu trúc và miền giá trị quy định (ví dụ: Độ tuổi phải là số nguyên dương >0 , Số điện thoại phải đủ số lượng ký tự).

7. ĐIỆN TOÁN ĐÁM MÂY VÀ DỮ LIỆU LỚN

Điện toán Đám mây (Cloud Computing) và Dữ liệu Lớn (Big Data) là hai trụ cột công nghệ luôn song hành và bổ trợ lẫn nhau trong thời đại số.

Mối quan hệ giữa hai khái niệm này có thể gói gọn trong một câu: **Big Data** cung cấp "giá trị tri thức", còn **Cloud Computing** cung cấp "môi trường và sức mạnh tính toán" để hiện thực hóa giá trị đó.



Các Thành phần của một Hệ sinh thái Big Data trên Cloud

Một kiến trúc Big Data hiện đại triển khai trên Đám mây thường bao gồm 5 tầng công nghệ:

- **Tầng Thu thập (Data Ingestion):**
 - Thu thập dữ liệu theo thời gian thực (Streaming) hoặc theo lô (Batch).
 - *Công nghệ Cloud:* AWS Kinesis, GCP Pub/Sub, Azure Event Hubs, Apache Kafka trên Cloud.
- **Tầng Lưu trữ (Data Storage):**
 - Lưu trữ dữ liệu thô (Data Lake) hoặc dữ liệu đã qua xử lý (Data Warehouse / Lakehouse).
 - *Công nghệ Cloud:* Amazon S3, Google Cloud Storage, Azure Blob Storage.
- **Tầng Xử lý & Biến đổi (Data Processing - ETL/ELT):**
 - Biến đổi, làm sạch và tổng hợp dữ liệu.
 - *Công nghệ Cloud:* AWS EMR, GCP Dataproc, Databricks, AWS Glue.
- **Tầng Lưu trữ Phân tích (Analytical Data Warehouse):**
 - CSDL truy vấn siêu tốc phục vụ báo cáo và phân tích.
 - *Công nghệ Cloud:* Google BigQuery, Snowflake, Amazon Redshift, Azure Synapse Analytics.
- **Tầng Khai thác (Analytics, BI & AI/ML):**
 - Trực quan hóa dữ liệu và huấn luyện mô hình Trí tuệ nhân tạo.
 - *Công nghệ Cloud:* Tableau, Power BI, Looker, Amazon SageMaker, GCP Vertex AI.

Đặc trưng của dữ liệu lớn

- **Dữ liệu lớn (Big Data)** không chỉ đơn thuần là dữ liệu có kích thước "rất lớn", mà được xác định dựa trên một tập hợp các thuộc tính đặc trưng giúp phân biệt nó với các hệ thống dữ liệu truyền thống.
- Mô hình phổ biến nhất để định nghĩa Big Data ban đầu là **mô hình 3V** (do Gartner đề xuất năm 2001), sau đó được phát triển rộng rãi thành **mô hình 5V** và hiện nay là **mô hình 7V** trong kỷ nguyên Cloud và Trí tuệ nhân tạo.

Đặc trưng Cốt lõi Ban đầu (Mô hình 3V)

- **Volume (Kích thước / Khối lượng):**

- ✓ Dữ liệu sinh ra với quy mô khổng lồ, vượt xa khả năng lưu trữ và xử lý của các hệ thống CSDL quan hệ truyền thống (RDBMS).
- ✓ Quy mô dữ liệu tính bằng Terabyte (TB), Petabyte (PB), Exabyte (EB) cho đến Zettabyte (ZB).
Nguồn gốc đến từ giao dịch thương mại, log máy chủ, thiết bị IoT, mạng xã hội.

- **Velocity (Tốc độ):**

- ✓ Dữ liệu được tạo ra, truyền tải và xử lý với tốc độ cực nhanh, nhiều trường hợp yêu cầu xử lý theo thời gian thực (Real-time) hoặc cận thời gian thực (Near real-time).
- ✓ *Ví dụ:* Dữ liệu quét thẻ tín dụng để phát hiện gian lận ngân hàng, dữ liệu phát hiện đe dọa an ninh mạng (SIEM), dòng dữ liệu cảm biến xe tự hành.

- **Variety (Tính đa dạng):**

- Dữ liệu không còn gói gọn trong các bảng dòng/cột ngăn nắp mà đến từ vô số định dạng khác nhau:
 - ✓ **Cấu trúc (Structured):** Dữ liệu SQL, bảng tính CSV.
 - ✓ **Bán cấu trúc (Semi-structured):** Dữ liệu JSON, XML, NoSQL logs.
 - ✓ **Phi cấu trúc (Unstructured):** Văn bản tự do, hình ảnh, video, âm thanh, bài viết mạng xã hội.

Các Đặc trưng Mở rộng (Mô hình 5V & 7V)

- **Veracity (Tính xác thực / Độ tin cậy):**

- ✓ Đề cập đến chất lượng, tính chính xác và độ đáng tin cậy của dữ liệu. Dữ liệu lớn thường chứa rất nhiều "nhiều" (noise), bản ghi bị thiếu, thông tin sai lệch hoặc dữ liệu giả mạo. Việc kiểm soát *Veracity* giúp tránh hiện tượng "đầu vào rác thì đầu ra rác" (Garbage-in, Garbage-out).

- **Value (Giá trị):**

- ✓ Đây là đặc trưng **quan trọng nhất** về mặt kinh tế. Khối lượng dữ liệu khổng lồ sẽ trở nên vô giá trị nếu không thể trích xuất ra được các thông tin chuyên sâu (Insights) giúp doanh nghiệp đưa ra quyết định tối ưu, tăng doanh thu hoặc dự báo rủi ro.

- **Variability (Tính biến động / Thất thường):**

- ✓ Sự thay đổi liên tục về nghĩa, ngữ cảnh hoặc cấu trúc dữ liệu theo thời gian (Schema Drift). Tốc độ và luồng dữ liệu cũng có thể bùng nổ theo đỉnh điểm (Peak hours/events) rồi giảm đột ngột (ví dụ: lượng bài đăng trong một sự kiện thể thao lớn).

- **Visualization (Tính trực quan hóa):**

- ✓ Khả năng chuyển đổi các tập dữ liệu phức tạp, đa chiều thành các biểu đồ, mô hình trực quan dễ hiểu giúp các nhà quản trị và người dùng phi kỹ thuật có thể nắm bắt bức tranh tổng thể một cách nhanh chóng.

So sánh Kiến trúc Big Data Truyền thống vs. Big Data trên Cloud

Tiêu chí	Big Data On-Premise (Truyền thống)	Big Data trên Cloud Computing
Chi phí ban đầu (CAPEX)	Rất cao (Mua máy chủ, tủ Racks, phòng Server, điện/làm mát)	Bằng 0 (Chuyển sang chi phí vận hành OPEX)
Khả năng mở rộng	Khó khăn, mất hàng tuần/tháng để mua thêm phần cứng	Tự động mở rộng (Auto-scaling) trong vài giây
Bảo trì & Vận hành	Tốn đội ngũ Kỹ sư IT/SysAdmin lớn để bảo trì cụm Hadoop/Spark	Nhà cung cấp Cloud lo toàn bộ hạ tầng (Managed Services)
Công nghệ & Cập nhật	Chậm chạp, khó cập nhật phần mềm mới	Bắt kịp ngay các công nghệ AI/Big Data mới nhất
Bảo mật & Tuân thủ	Tự quản lý toàn bộ hệ thống bảo mật	Hưởng lợi từ tiêu chuẩn an ninh quốc tế của nhà cung cấp Cloud

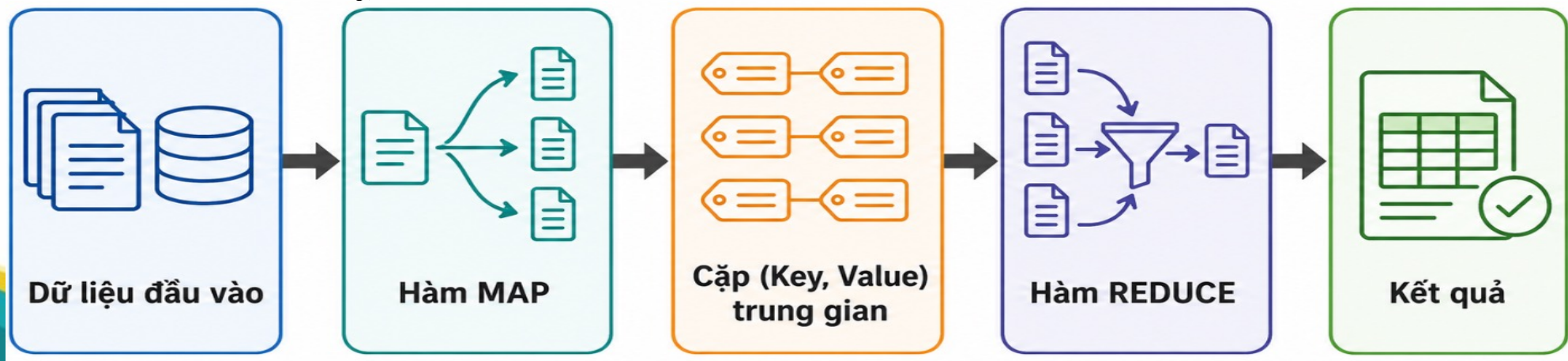
MapReduce ở mức khái niệm

- **MapReduce** là một mô hình lập trình (programming model) và là một hạ tầng phần mềm được Google giới thiệu nhằm xử lý và tạo ra các tập dữ liệu cực kỳ lớn (Big Data) một cách tự động, song song và phân tán trên các cụm máy tính (clusters) gồm hàng ngàn máy chủ thông thường.
- Ở mức độ khái niệm, MapReduce giải quyết một bài toán phức tạp bằng chiến lược "**Chia để trị**" (**Divide and Conquer**): Chia nhỏ một tập dữ liệu khổng lồ thành nhiều phần nhỏ để xử lý song song trên nhiều máy tính, sau đó tổng hợp các kết quả trung gian lại thành kết quả cuối cùng.

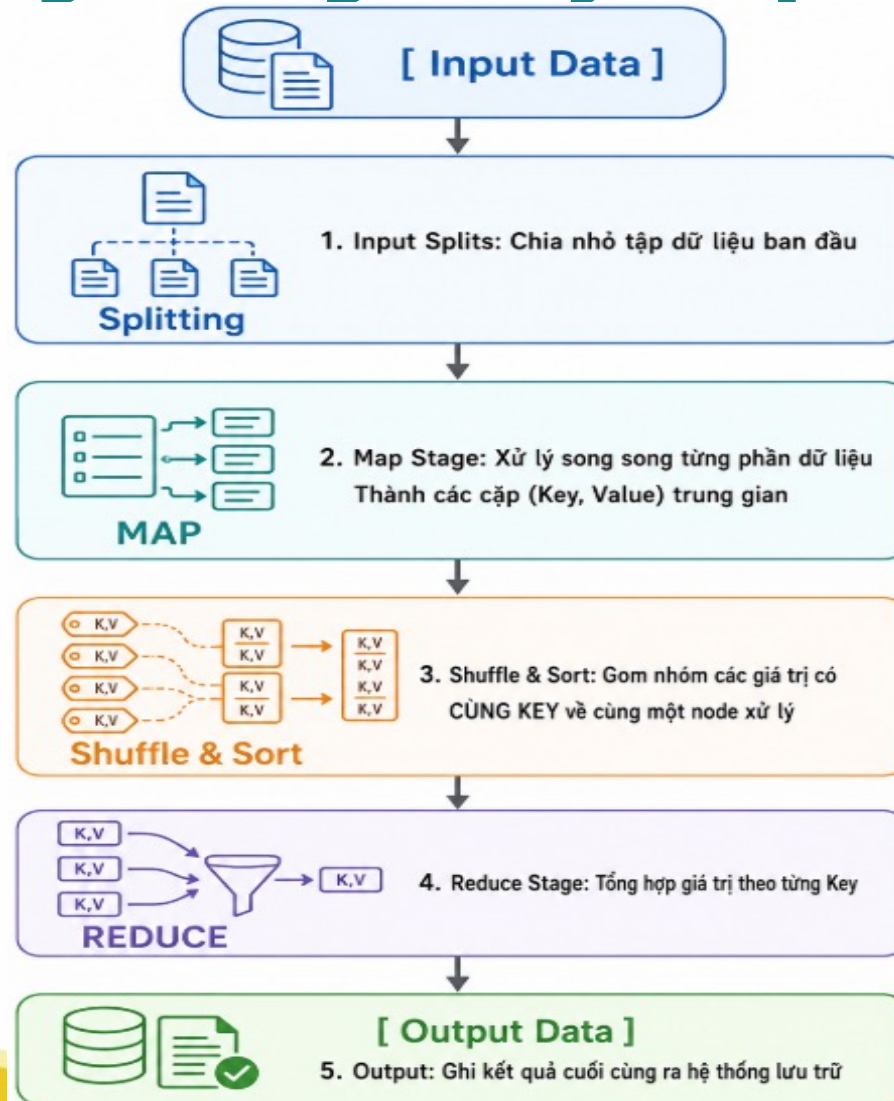
MapReduce ở mức khái niệm

Ý tưởng Cốt lõi: Hai Hàm "Map" và "Reduce"

- Ý tưởng của MapReduce xuất phát từ các hàm xử lý danh sách trong lập trình chức năng (Functional Programming). Toàn bộ luồng xử lý dữ liệu dựa trên các cặp **Key-Value** (Khoá - Giá trị).



Các Bước Trong Luồng Xử lý MapReduce Toàn diện



Apache Hadoop

- **Apache Hadoop** là một framework mã nguồn mở (Open-source Framework) viết bằng Java, cho phép lưu trữ và xử lý các tập dữ liệu cực kỳ lớn (Big Data) một cách phân tán trên các cụm máy tính (clusters) gồm hàng trăm đến hàng ngàn máy chủ phổ thông (commodity hardware).
- Nếu như **MapReduce** cung cấp *tư duy và thuật toán xử lý*, thì **Hadoop** chính là *hệ sinh thái hoàn chỉnh* giúp hiện thực hóa tư duy đó vào thực tế.

Apache Hadoop

HADOOP ECOSYSTEM

1.



Hadoop MapReduce / Spark

(Tầng Xử lý / Processing)

2.



YARN

(Tầng Quản lý Tài nguyên)

3.



HDFS

(Tầng Lưu trữ Phân tán)

4.



Hadoop Common

(Thư viện & Utilities)

Apache Hadoop

- Hadoop kết hợp HDFS, YARN và MapReduce trong hệ sinh thái dữ liệu lớn
- Mạnh trong xử lý theo lô và lưu trữ phân tán
- Có nhiều công cụ đi kèm trong hệ sinh thái
- Hiện nay thường được thay thế hoặc bổ sung bằng dịch vụ cloud managed và Spark

Apache Spark

- **Apache Spark** là một engine tính toán phân tán (distributed computing engine) mã nguồn mở, được thiết kế để xử lý dữ liệu lớn (Big Data) với tốc độ cực nhanh nhờ cơ chế **xử lý trên bộ nhớ trong (In-Memory Processing)**.
- Nếu như Apache Hadoop MapReduce là thế hệ Big Data thứ nhất (phụ thuộc vào đọc/ghi ổ đĩa Disk I/O), thì Apache Spark chính là chuẩn mực của thế hệ thứ hai — giải quyết hoàn hảo bài toán về hiệu năng và độ trễ.

Apache Spark

- Spark xử lý dữ liệu phân tán với khả năng xử lý trong bộ nhớ
- Hỗ trợ batch, streaming, SQL, machine learning và graph processing
- Thường nhanh và linh hoạt hơn MapReduce truyền thống
- Có thể chạy trên cluster cloud hoặc dịch vụ được quản lý

Xử lý theo lô và xử lý dòng

- Trong kỹ thuật dữ liệu và kiến trúc hệ thống Big Data, **Xử lý theo lô (Batch Processing)** và **Xử lý dòng (Stream Processing / Real-time Processing)** là hai mô hình tính toán cơ bản giải quyết hai bài toán khác nhau về tốc độ, khối lượng và độ trễ của dữ liệu.

Xử lý theo lô và xử lý dòng

Xử lý theo lô (Batch Processing)

- **Khái niệm:** Dữ liệu được thu thập, tích lũy lại thành các tập lớn trong một khoảng thời gian (theo giờ, ngày, tuần, tháng) trước khi toàn bộ tập dữ liệu đó được đưa vào hệ thống để xử lý cùng một lúc.
- **Tập dữ liệu: Hạn định (Bounded Data)** — Dữ liệu có điểm bắt đầu và điểm kết thúc rõ ràng tại thời điểm xử lý.
- **Độ trễ (Latency):** Cao — Từ vài phút, vài giờ đến hàng ngày.

Xử lý dòng (Stream Processing)

- **Khái niệm:** Dữ liệu được xử lý ngay lập tức khi nó vừa được sinh ra (hoặc chỉ sau vài mili-giây đến vài giây). Hệ thống xử lý từng sự kiện (event-by-event) hoặc theo từng mẻ cực nhỏ (micro-batch) một cách liên tục.
- **Tập dữ liệu: Bất hạn (Unbounded Data)** — Luồng dữ liệu chảy liên tục không có điểm kết thúc.
- **Độ trễ (Latency):** Cực thấp — Tính bằng mili-giây (milliseconds) hoặc giây (seconds).

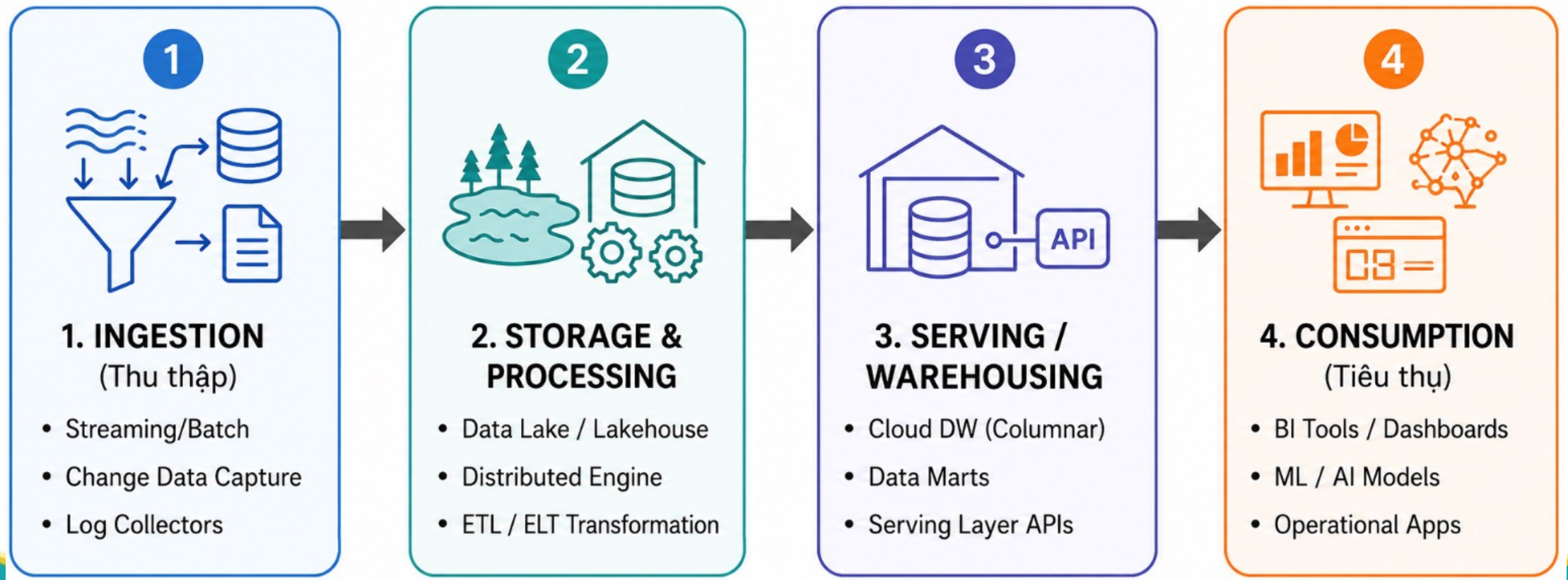
So sánh Chi tiết: Batch Processing vs. Stream Processing

Tiêu chí	Batch Processing (Xử lý theo lô)	Stream Processing (Xử lý dòng)
Bản chất dữ liệu	Bounded (Dữ liệu tĩnh, đã dừng thu thập)	Unbounded (Luồng dữ liệu động, chảy liên tục)
Độ trễ (Latency)	Cao (Vài phút $\rightarrow$ vài ngày)	Cực thấp (Vài mili-giây $\rightarrow$ vài giây)
Băng thông (Throughput)	Cực cao (Xử lý khối lượng Terabytes/Petabytes)	Vừa và lớn (Xử lý theo từng bản ghi/sự kiện)
Mô hình tính toán	Lập lịch theo chu kỳ (Cronjob, Airflow)	Chạy liên tục 24/7 (Event-driven)
Yêu cầu về trạng thái (State)	Không cần duy trì trạng thái lâu dài giữa các đợt	Cần quản lý trạng thái phức tạp (<i>Stateful Streaming</i>)
Phục hồi lỗi (Fault Tolerance)	Đơn giản: Chạy lại toàn bộ job bị lỗi	Phức tạp: Cần cơ chế <i>Checkpointing</i> , <i>WAL</i> , <i>Exactly-once semantics</i>
Công cụ phổ biến	Apache Hadoop MapReduce, Apache Spark (Batch API), Snowflake, BigQuery	Apache Flink, Apache Kafka / Spark Streaming, AWS Kinesis
Ứng dụng điển hình	Báo cáo tài chính cuối tháng, huấn luyện mô hình ML hàng tuần, sao lưu dữ liệu	Phát hiện gian lận thẻ tín dụng, cảnh báo an ninh mạng (SIEM), theo dõi xe ô tô công nghệ

Pipeline dữ liệu trên cloud

- **Pipeline dữ liệu trên Cloud (Cloud Data Pipeline)** là một hệ thống tự động hóa quá trình di chuyển, biến đổi và quản lý dữ liệu từ nhiều nguồn khác nhau (như CSDL operational, thiết bị IoT, logs, SaaS APIs) về các hạ tầng lưu trữ phân tích (Data Warehouse, Data Lake, Data Lakehouse) để phục vụ cho báo cáo BI, Phân tích nâng cao hoặc huấn luyện AI/ML.
- Trên hạ tầng Cloud, pipeline dữ liệu tận dụng triệt để các đặc tính **Serverless, Auto-scaling (Tự động mở rộng), Tách biệt Compute/Storage (Tách biệt tính toán & lưu trữ)** để đạt hiệu năng tối ưu với chi phí linh hoạt nhất.

Pipeline dữ liệu trên cloud



So sánh Bộ Công cụ Pipeline trên các Nền tảng Cloud hàng đầu

Tầng chức năng	AWS (Amazon Web Services)	Google Cloud (GCP)	Microsoft Azure	Multi-Cloud / Third-party
Streaming Ingestion	Amazon Kinesis, MSK (Kafka)	Pub/Sub	Event Hubs	Apache Kafka, Confluent
Batch/CDC Ingestion	AWS DMS, AppFlow	Datastream, Cloud Data Fusion	Azure Data Factory (ADF)	Fivetran, Airbyte
Data Lake Storage	Amazon S3	Google Cloud Storage (GCS)	Azure Data Lake Storage (ADLS Gen2)	MinIO, Ceph
Data Processing	AWS Glue, EMR (Spark)	Cloud Dataflow (Beam), Dataproc	Azure Synapse / Databricks	Apache Spark, dbt
Data Warehouse / Lakehouse	Amazon Redshift, Athena	BigQuery	Azure Synapse Analytics	Snowflake, Databricks
Orchestration (Điều phối)	AWS Step Functions, Managed Airflow (MWAA)	Cloud Composer (Airflow)	Azure Data Factory Pipelines	Apache Airflow, Dagster, Prefect

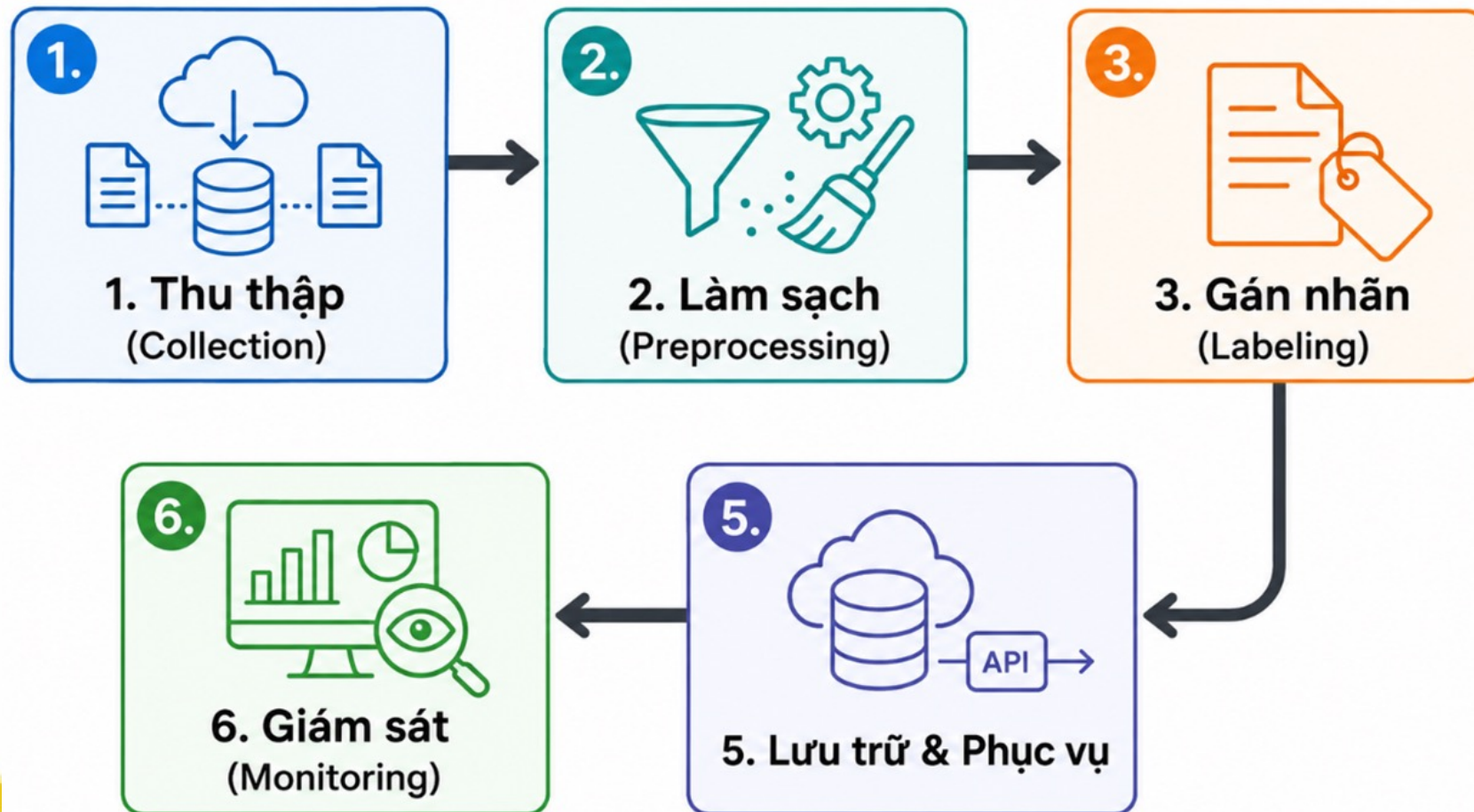
Dữ liệu phục vụ AI và học máy

- Trong kỷ nguyên trí tuệ nhân tạo (AI) và Học máy (Machine Learning/Deep Learning), **dữ liệu đóng vai trò là "nhiên liệu" quyết định sự thành bại của mô hình**. Một thuật toán AI tiên tiến đến đâu nhưng luyện trên dữ liệu kém chất lượng cũng sẽ cho ra kết quả sai lệch — đúng theo nguyên lý kinh điển **"Garbage in, Garbage out" (GIGO)**.
- Khác với pipeline phân tích dữ liệu truyền thống (phục vụ BI/Báo cáo vốn chủ yếu làm việc với dữ liệu cấu trúc), **AI Data Pipeline** phải giải quyết các tập dữ liệu phi cấu trúc không lồ (văn bản, hình ảnh, âm thanh, video) và đòi hỏi quy trình xử lý khắt khe hơn rất nhiều.

Dữ liệu phục vụ AI và học máy

- AI cần dữ liệu đủ lớn, sạch, đại diện và có nhãn phù hợp
- Pipeline cần chuẩn bị dữ liệu huấn luyện, kiểm thử và suy luận
- Feature store giúp quản lý đặc trưng dùng lại
- Dữ liệu phải được kiểm soát quyền riêng tư và chi phí xử lý

Vòng đời Dữ liệu AI/ML (AI Data Lifecycle)



Cơ sở dữ liệu vector và tìm kiếm ngữ nghĩa

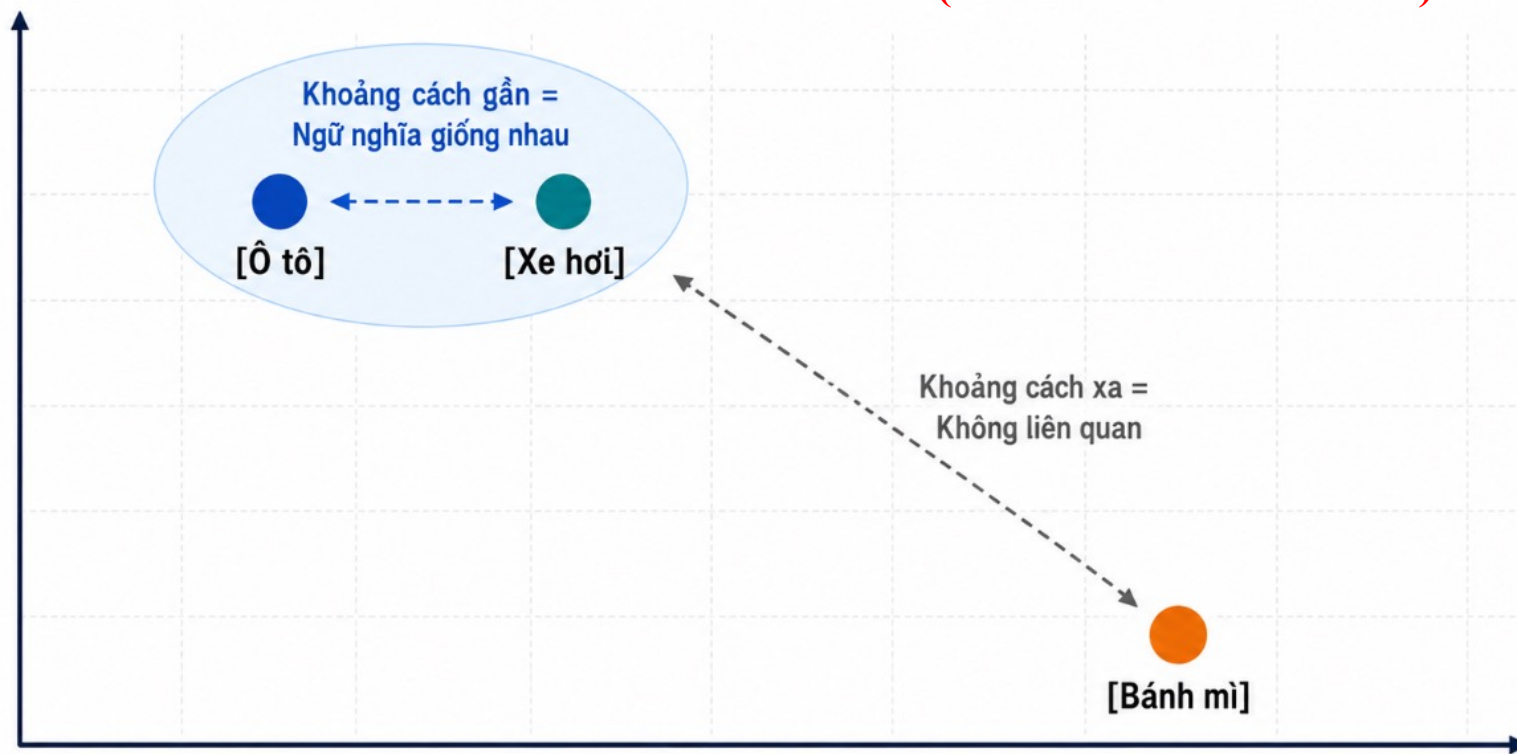
- Cơ sở dữ liệu Vector (Vector Database) và Tìm kiếm ngữ nghĩa (Semantic Search) là bộ đôi công nghệ cốt lõi đứng sau làn sóng Generative AI và các ứng dụng Mô hình ngôn ngữ lớn (LLM) hiện nay.
- Chúng giải quyết triệt để hạn chế của các hệ thống tìm kiếm truyền thống: hiểu được "ý nghĩa" và "ngữ cảnh" đằng sau câu từ chứ không chỉ khớp các từ khóa (keywords) đơn thuần.

Cơ sở dữ liệu vector và tìm kiếm ngữ nghĩa

- Văn bản, hình ảnh hoặc âm thanh có thể được mã hóa thành vector
- Tìm kiếm tương đồng giúp truy vấn theo ý nghĩa, không chỉ từ khóa
- Quan trọng trong chatbot, tìm kiếm tài liệu và RAG
- Cần kết hợp bảo mật dữ liệu, kiểm soát truy cập và đánh giá chất lượng

Cơ sở dữ liệu vector và tìm kiếm ngữ nghĩa

KHÔNG GIAN VECTO' NHIỀU CHIỀU (EMBEDDING SPACE)



● Mỗi điểm = một vectơ biểu diễn ngữ nghĩa

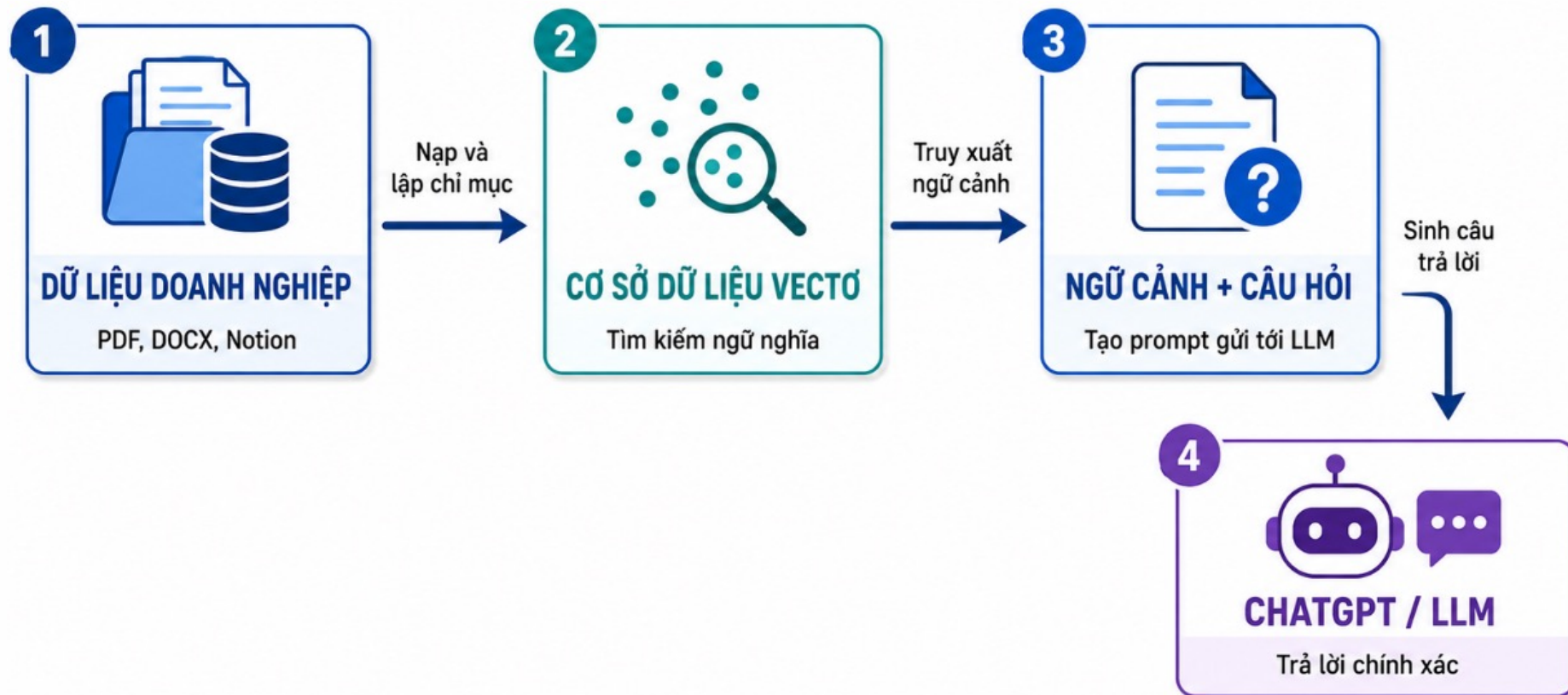
Hệ sinh thái Công cụ Vector Database Hiện nay

Thị trường hiện chia làm 2 nhóm chính:

Dòng sản phẩm	Đại diện điển hình	Ưu điểm	Trường hợp sử dụng
Native Vector DB <i>(Thành lập chuyên biệt cho Vector)</i>	Pinecone, Milvus, Qdrant, ChromaDB, Weaviate	Tối ưu hóa tuyệt đối cho vector, hiệu năng cực cao, hỗ trợ mở rộng ngang (Scale-out) phân tán tốt.	Các ứng dụng AI quy mô lớn, tìm kiếm thời gian thực với hàng chục triệu bản ghi.
Vector Extensions <i>(Tích hợp vào CSDL có sẵn)</i>	pgvector (PostgreSQL), Elasticsearch, MongoDB Atlas, Redis	Tận dụng hạ tầng CSDL sẵn có, dễ dàng Join dữ liệu vector với dữ liệu quan hệ (Metadata Filtering).	Doanh nghiệp muốn tích hợp AI vào hệ thống hiện hữu mà không muốn vận hành CSDL mới.

Ứng dụng Thực tế Cốt lõi: Kiến trúc RAG (Retrieval-Augmented Generation)

- Ứng dụng nổi bật nhất của Vector DB và Semantic Search chính là giải quyết bài toán "Ảo giác" (Hallucination) và giới hạn tri thức của các mô hình LLM thông qua kiến trúc RAG:



i RAG = Retrieval-Augmented Generation (Sinh tăng cường bằng truy xuất)

Thank you!

